# CEH v13 Module 13: Hacking Web Servers

CEH v13 AI (312-50) Exam-Focused Study Guide

Generated study material - not official EC-Council content

# Table of Contents

1. Module at a Glance
2. Big Picture Map
3. Topic-by-Topic Guide
   3.1 Learning Objectives of Module 13
   3.2 Web Server Security Issues
   3.3 Apache Web Server Architecture
   3.4 Apache Vulnerabilities
   3.5 IIS Web Server Architecture
   3.6 IIS Vulnerabilities
   3.7 Nginx Web Server Architecture
   3.8 Nginx Vulnerabilities
   3.9 DNS Server Hijacking
   3.10 Directory Traversal Attacks
   3.11 Web Server Misconfiguration
   3.12 HTTP Response-Splitting Attack
   3.13 Web Cache Poisoning Attack
   3.14 SSH Brute Force Attack
   3.15 HTTP/2 Continuation Flood Attack
   3.16 Frontjacking Attack
   3.17 Other Web Server Attacks
   3.18 Web Server Attack Methodology
   3.19 Information Gathering from Robots.txt File
   3.20 Web Server Footprinting / Banner Grabbing
   3.21 IIS Information Gathering using Shodan
   3.22 Abusing Apache mod_userdir to Enumerate User Accounts
   3.23 Directory Brute Forcing
   3.24 Vulnerability Scanning
   3.25 Nginx Vulnerability Scanning using Nginxpwner
   3.26 Finding Exploitable Vulnerabilities (Exploit-DB)
   3.27 Path Traversal via Misconfigured NGINX Alias
   3.28 Countermeasures: Patches and Updates
   3.29 Countermeasures: Protocols and Accounts
   3.30 Countermeasures: Files and Directories
   3.31 How to Defend against Web Server Attacks (Ports, Machine.config, Certificates, Code Access Security)

3.32 How to Defend against Web Server Attacks (Cont'd) - Shares, Permissions, Script Mappings
3.33 How to Defend against Web Server Attacks (Cont'd) - General Hardening Practices
3.34 How to Defend against HTTP Response-Splitting and Web Cache Poisoning
3.35 How to Defend against DNS Hijacking
3.36 Web Server Security Tools
3.37 Web Server Pen Testing Tools
3.38 Module Summary
4. Tool Encyclopedia
5. Command Sheet
5.1 Netcat manual banner grab
5.2 Telnet manual banner grab
5.3 Nmap http-userdir-enum (initial scan)
5.4 Nmap http-userdir-enum (custom wordlist)
5.5 Nmap http-brute with custom User-Agent
5.6 Nginxpwner scan
5.7 Dirhunt directory crawl
6. Ports, Protocols, Standards and Acronyms
7. Attack vs Countermeasure and Look-alike Comparisons
7.1 Attack vs Countermeasure
7.2 Look-alike Comparisons
8. Exam Cheat Sheet
9. Practice Questions
10. Flashcards
11. Glossary
12. Coverage Report

Source: 20 pages of screenshots from the official Module 13 theory videos (slides 2 to 50). Content marked "ADDED CONTEXT" comes from outside the screenshots. Items marked "verify" should be checked against current official sources. Items marked [UNCLEAR on page N] were too small or blurry to read with full confidence.

# 1. Module at a Glance

**Module:** 13 - Hacking Web Servers (EC-Council CEH v13 AI).

**What this module is (5 lines):**

1. It explains why web servers get hacked and walks through the inside architecture of the three big web servers: Apache, IIS, and Nginx.
2. It lists real, named CVE-style vulnerabilities for each web server so you learn to connect a server type to its known weak spots.
3. It covers the main attack techniques: DNS server hijacking, directory traversal, misconfiguration abuse, HTTP response splitting, web cache poisoning, SSH brute force, HTTP/2 continuation flood, and FrontJacking.
4. It gives a full attack methodology: information gathering, footprinting, mirroring, vulnerability scanning, and password hacking, each with real tools (Netcat, Telnet, Shodan, Nmap, Dirhunt, Acunetix, Nginxpwner, Exploit-DB, Kyubi).
5. It closes with a long, detailed set of countermeasures and a list of security and pen-testing tools.

**Why it matters:** A web server is reachable from anywhere on the internet, which is exactly why it is attacked more than internal systems. This module sits between network-level attacks (Modules 3 and 4) and application-level attacks (Module 14), and the exam expects you to tell all three apart. Expect several "which attack is this" and "which tool fits this job" questions from this module.

**What the exam usually asks from it:**

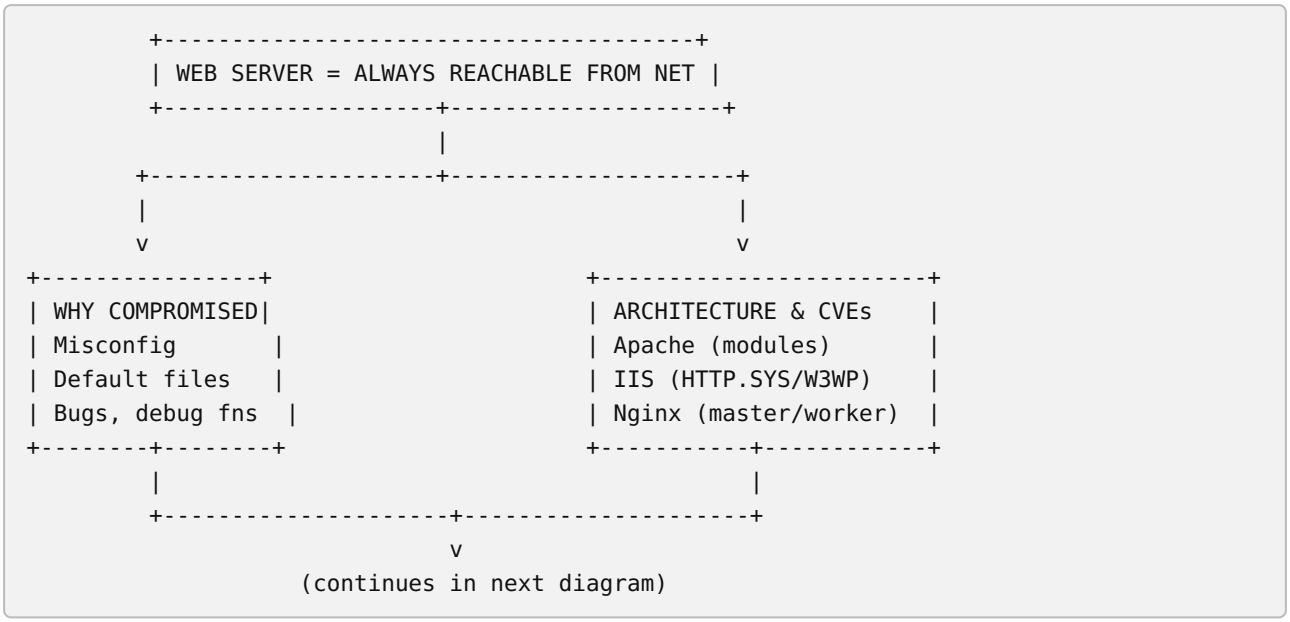
- Why web servers get compromised (misconfiguration, default files, bugs).
- Matching a CVE-style description to an attack name (response splitting, cache poisoning, continuation flood, FrontJacking).
- Reading a banner-grabbing output (Netcat or Telnet) to identify the web server and version from the `Server:` header.
- Choosing the right Nmap script and argument for user enumeration (`http-userdir-enum`, `userdir.users`, `http.useragent`).
- Matching a tool to a job: Shodan (device search), Dirhunt (directory crawl without brute force), Nginxpwner (Nginx misconfig scan), Exploit-DB (public exploit search), Kyubi (Nginx alias traversal).
- Picking the right countermeasure for a described attack or misconfiguration.
- The six steps of the web server attack methodology, in order.

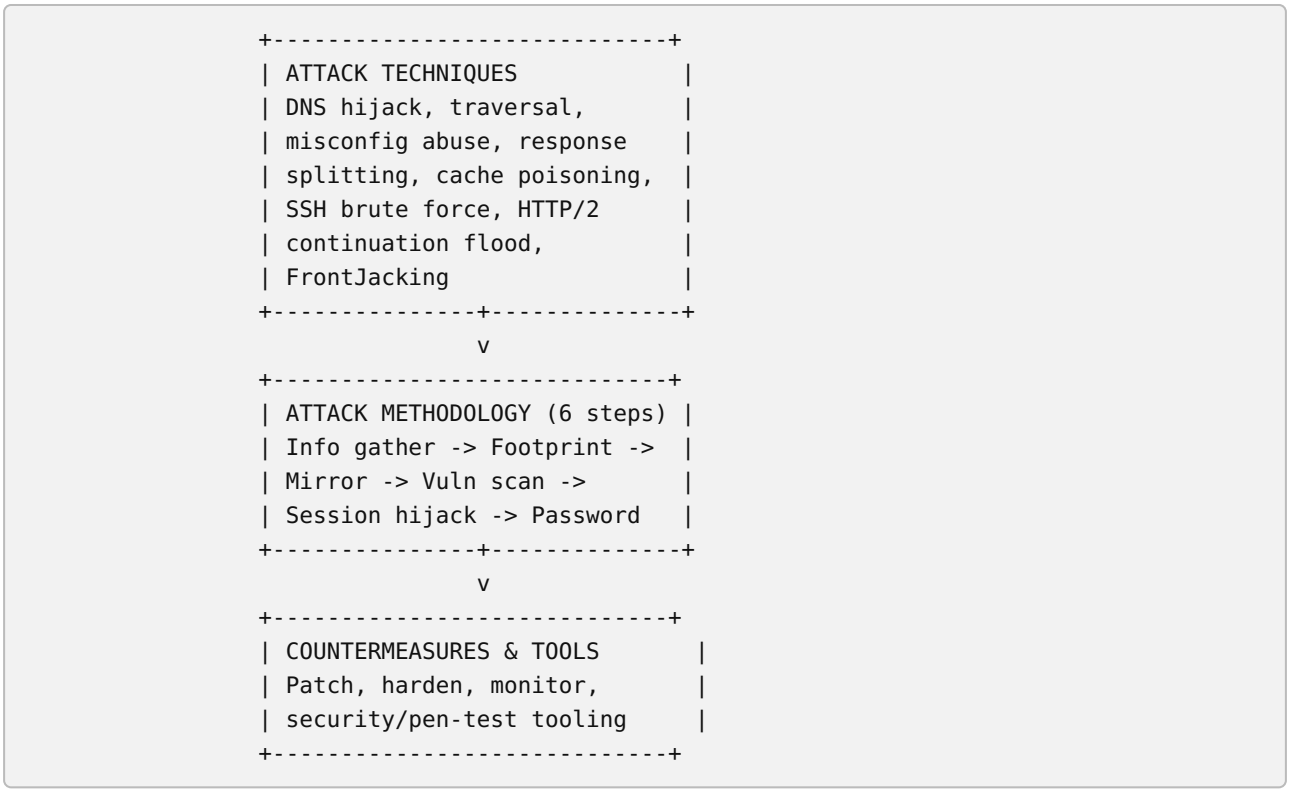
**The 10 things to remember most:**

1. Web servers are hacked mostly through misconfiguration and leftover default files, not just through zero-day code bugs.
2. Apache, IIS, and Nginx each have a different internal architecture: Apache uses modules (`mod_ssl`, `mod_auth`, `mod_rewrite`, `mod_proxy`); IIS splits work between kernel-mode `HTTP.SYS` and user-mode `W3WP.exe`; Nginx uses one master process and many worker processes.
3. Directory traversal uses `../` (dot-dot-slash) to escape the web root.
4. HTTP response splitting injects CRLF (`%0d%0a`) into input so the server sends back two HTTP responses instead of one.
5. Web cache poisoning tricks a cache into storing an attacker's page under the URL of a legitimate page.
6. The HTTP/2 continuation flood attack sends endless CONTINUATION frames after a HEADERS frame without the `END_HEADERS` flag, which exhausts server memory. This matches real CVE-2024-27316 on Apache.
7. FrontJacking targets a misconfigured Nginx reverse proxy in shared hosting using CRLF injection, header injection, and XSS together.
8. The web server attack methodology has six steps: information gathering, web server footprinting, website mirroring, vulnerability scanning, session hijacking, and web server password hacking.
9. Banner grabbing with Netcat or Telnet on port 80 reveals the `Server:` header (for example `Microsoft-IIS/10.0`), which tells the attacker the exact web server software.
10. robots.txt, meant to guide search engine crawlers, often leaks the directory structure an attacker should not see.

## 2. Big Picture Map

Everything in this module fits one flow: a web server exists because it answers requests from anywhere on the internet; that same openness is why it gets attacked; attackers follow a methodology to find and use a weakness; defenders answer with hardening at every layer.




How each attack maps to its usual target:

| Attack | Usual target server | Core weakness used |
|---|---|---|
| DNS server hijacking | DNS server in front of web server | Weak admin access on DNS |
| Directory traversal | Any (IIS example in notes) | Missing input validation |
| Misconfiguration abuse | Apache, Nginx, IIS | Verbose default settings |
| HTTP response splitting | Apache (example in notes) | Unvalidated header input |
| Web cache poisoning | Any with a front cache | Unvalidated cache key input |

| Attack | Usual target server | Core weakness used |
|---|---|---|
| SSH brute force | SSH service on the server | Weak or reused passwords |
| HTTP/2 continuation flood | Apache (CVE-2024-27316) | No limit on CONTINUATION frames |
| FrontJacking | Nginx reverse proxy | Trusting the Host header |

# 3. Topic-by-Topic Guide

The topics below follow the order of the screenshots.

## 3.1 Learning Objectives of Module 13

**What it is:** The opening slide lists four goals: summarize web server concepts, demonstrate web server attacks, explain the attack methodology, and explain countermeasures. These four goals are also the skeleton of this guide.

**EXAM FOCUS:** Expect at least one question drawn from each of the four objective areas: concepts, attacks, methodology, countermeasures.

**Memory trick:** "Concepts, Combat, Course, Cure" -> web server Concepts, attack techniques (Combat), the attack methodology (Course of action), and Countermeasures (Cure).

## 3.2 Web Server Security Issues

**What it is (simple words):** Web servers face attacks mostly because they must be reachable from the public internet. Network-level problems can be solved by firewalls and IDS, but a web server still has to accept connections from strangers, so a different set of weaknesses matters: how the server itself is set up.

**Analogy:** A bank vault behind locked doors is safe. A bank's drive-through teller window has to stay open to the street for customers, so it needs its own extra protections (bulletproof glass, cameras, limits on cash handed over). The web server is the drive-through window of the network.

**Why web servers are compromised (from the notes):**

1. Improper file and directory permissions.
2. Unnecessary default, backup, or sample files left on the server.
3. Misconfigurations in the web server, operating system, and network.
4. Bugs in the server software, OS, and web applications.
5. Administrative or debugging functions left enabled or reachable.
6. Use of self-signed certificates, misconfigured SSL, default certificates, and weak encryption.

7. Not using a dedicated server for web services (mixing roles).

**Impact of web server attacks (from the notes):**

1. Compromise of user accounts.
2. Website defacement.
3. Secondary attacks launched from the compromised website.
4. Root access gained on other applications or servers.
5. Data tampering and data theft.
6. Reputational damage to the company.

**How it works (numbered steps, generalized):**

1. An attacker finds a server exposed on the internet.
2. The attacker looks for one of the seven weaknesses above (misconfig, leftover files, bugs, debug functions, weak certs, shared server role).
3. The attacker exploits the weakness to get a foothold.
4. The attacker uses that foothold to cause one or more of the six impacts.

**Real-world use:** A small business sets up a Windows Server box that runs both the company website (IIS) and the file server. The IIS install keeps its default sample pages and an unused FTP service. An attacker finds the sample pages through a search engine, learns the IIS version, and finds a public exploit for that version. Because the same box also holds company files, the attacker gets both the website and the files.

**Tools and commands used here:** None specific on this slide; the rest of the module supplies the tools.

**AI angle:** ADDED CONTEXT: an attacker can ask an AI chatbot to list common default files and misconfiguration patterns for a named web server version, speeding up the search. A defender can use the same kind of prompt to build a hardening checklist. Example prompt: "List common default files, sample pages, and misconfigurations found on a fresh Apache 2.4 install that an attacker might look for."

**Countermeasures and defense:** Remove default/sample/backup files, apply least-privilege file permissions, patch regularly, disable unneeded admin/debug endpoints, use proper CA-signed certificates, and run web services on a dedicated server, not a shared one.

> **EXAM TIP:**If a question asks "why is a web server more exposed than an internal database server," the answer is reachability from the public internet, not a difference in operating system.
> **TRAP:** Firewalls and IDS are listed as solving network and OS level attacks. A question may try to make you think a firewall alone secures a web server; the slide explicitly says this is not enough because the web server must stay reachable.

**EXAM FOCUS:** Keywords "default files", "sample files", "debug functions enabled" point to misconfiguration-class answers. Keywords "reputational damage", "defacement" point to impact-class answers, not cause-class answers; read the question carefully to see if it asks for a cause or a result.

**Memory trick:** Causes = "MUDBAD": Misconfig, Unnecessary default files, Debug/admin functions, Bugs, Access/permission errors, Dedicated server missing. Impacts = "CARDS": Compromise accounts, Attacks launched from site, Root access elsewhere, Data theft, Standing/reputation hit.

## 3.3 Apache Web Server Architecture

**What it is (simple words):** Apache HTTP Server is a free, open-source program that answers web requests. It is built from small plug-in pieces called modules, which can be turned on or off to add features.

**Analogy:** Apache is like a smartphone. The core phone makes calls, and you add apps (modules) for everything else: a camera app (`mod_ssl` for encryption), a lock-screen app (`mod_auth` for authentication), a redirect app (`mod_rewrite` for URL rewriting), and a messenger app that forwards chats to someone else (`mod_proxy` for forwarding requests to another server).

**How it works (numbered steps):**

1. An HTTP client sends an HTTP or HTTPS request to the Apache web server.
2. Apache passes the request through its loaded modules, for example `mod_ssl` (TLS/SSL encryption and decryption), `mod_auth` (authentication checks), `mod_rewrite` (URL rewriting rules), and `mod_proxy` (forwarding requests to another server).
3. If the content is dynamic (built by code, not a static file), Apache forwards the request to an application server.
4. The application server retrieves or builds the dynamic content and returns it to Apache.
5. Apache sends the HTTP(S) response back to the client.

**Key terms:**

- **Module:** a plug-in piece of code that adds one feature to Apache without changing its core.
- **mod_ssl:** handles HTTPS encryption and decryption.
- **mod_auth:** handles authentication (checking who the user is).
- **mod_rewrite:** rewrites incoming URLs based on rules, often used for clean URLs or redirects.
- **mod_proxy:** lets Apache act as a proxy, forwarding requests to another server (such as an application server).
- [UNCLEAR on page 2]: the slide shows a box labeled "BMMM Extensible Agent" next to the HTTP server; the exact wording could not be confirmed from the image. Best reading, labelled uncertain: this is likely a diagram label for an

extensible module or agent component and is not a standard, documented Apache term.

**Real-world use:** An online store runs Apache with `mod_ssl` for checkout encryption, `mod_rewrite` to turn `product.php?id=5` into `/product/5`, and `mod_proxy` to forward requests for the shopping cart to a separate Java application server.

**Tools and commands used here:** None directly; architecture knowledge feeds into reading Apache CVEs in the next section.

**Countermeasures and defense:** Only enable the modules actually in use; disable or remove unused modules such as old authentication or status modules that expand the attack surface.

> **EXAM TIP:**Apache's biggest architectural idea for the exam is "modular". If a question describes a web server built from small pluggable pieces, matched against IIS's kernel/user mode split and Nginx's master/worker model, the modular one is Apache.

**EXAM FOCUS:** Expect a matching question: "Which module handles encryption?" (`mod_ssl`), "Which module forwards requests to another server?" (`mod_proxy`), "Which module rewrites URLs?" (`mod_rewrite`).

**Memory trick:** "SARP": SSL (mod_ssl), Auth (mod_auth), Rewrite (mod_rewrite), Proxy (mod_proxy). Say it as "Apache uses SARP."

## 3.4 Apache Vulnerabilities

**What it is (simple words):** A list of real, named weaknesses found in Apache HTTP Server and in other software published under the Apache Software Foundation (Submarine, Allura, OFBiz). Knowing the pattern of each vulnerability type matters more than memorizing exact CVE numbers.

**How it works / vulnerability list (from the notes):**

- **HTTP response splitting:** bad input validation lets an attacker inject headers into the response, leading to XSS, cache poisoning, or data leaks.
- **HTTP/2 DoS, endless CONTINUATION frames:** attacker sends endless HTTP/2 headers; server memory fills up and the server stops responding or crashes.
- **mod_macro buffer over-read:** mod_macro mishandles macro expansion; crafted requests read sensitive memory next to the buffer.
- **HTTP/2 DoS, initial window size 0:** attacker sets the HTTP/2 initial window size to 0, blocking data transfer and causing denial of service.
- **HTTP/2 stream memory not reclaimed on RST:** stream memory is not freed after a reset (RST); repeating this exhausts memory.
- **Apache Submarine insecure default config:** default admin credentials lead to remote code execution when attackers log in with them.

- **Apache Submarine improper authorization:** faulty authorization checks in core components allow unauthorized access or privilege escalation.
- **Apache Allura DNS rebinding:** attacker manipulates DNS responses in the import feature to reach internal services.
- **Apache OFBiz path traversal:** weak directory path limits let an attacker read files outside the intended directory, or run code.
- **Apache Submarine SQL injection:** improper handling of SQL input lets an attacker run arbitrary SQL queries.

> **ADDED CONTEXT:**The HTTP/2 DoS by memory exhaustion on endless CONTINUATION frames matches the real-world "HTTP/2 CONTINUATION Flood" issue tracked as CVE-2024-27316 for Apache HTTP Server (verify exact version range). The server buffers incoming HTTP/2 headers in the nghttp2 library to build an informative error response; if the client never stops sending CONTINUATION frames, that buffer keeps growing until the server runs out of memory (out-of-memory crash). Source: IBM security bulletin on CVE-2024-27316. This same mechanic reappears in section 3.15 as its own named attack.

**Real-world use:** A pentester targeting an Apache-based intranet portal finds it is Apache Submarine with default admin credentials still set. Logging in with the default username and password gives remote code execution immediately, without needing any further exploit.

**Tools and commands used here:** https://httpd.apache.org (official Apache HTTP Server site, named on the slide); https://cve.mitre.org (the CVE database referenced for looking up these vulnerabilities).

**Countermeasures and defense:** Patch Apache and any Apache-family product (Submarine, Allura, OFBiz) promptly; change default admin credentials before going live; validate all header and macro input; set sane HTTP/2 limits (minimum window size, maximum CONTINUATION frames per stream, timeouts on incomplete header blocks).

> **TRAP:**Not every item in this list is "Apache HTTP Server" itself. Submarine, Allura, and OFBiz are separate Apache Software Foundation projects. A question may ask "which of these affects the Apache HTTP Server directly" versus "which affects an Apache ecosystem project"; only the first five rows (response splitting through RST memory) are about the core HTTP server.

**EXAM FOCUS:** Keywords "endless CONTINUATION frames" or "memory exhaustion" -> HTTP/2 DoS pattern. Keywords "default admin credentials" -> insecure default configuration. Keywords "DNS responses", "import functionality" -> DNS rebinding in Allura.

**Memory trick:** Group by result: "DoS trio" (endless CONTINUATION, window size 0, RST memory) all cause denial of service on HTTP/2; "RCE pair" (default creds, SQLi) both give the attacker code or data control.

## 3.5 IIS Web Server Architecture

**What it is (simple words):** Internet Information Services (IIS) is Microsoft's web server for Windows Server. It answers browser requests by splitting work between the Windows kernel (fast, low-level) and user-mode worker processes (flexible, where your application code runs).

**Analogy:** Think of IIS as an airport. The kernel-mode HTTP.SYS is the runway and air traffic control: it receives every incoming plane (request) and knows where to send it, very fast and very low-level. The user-mode side (Windows Activation Service, W3WP.exe worker process) is the terminal building: it is where passengers (requests) actually get processed, checked in, and handled by staff (application code).

**How it works (numbered steps, kernel then user mode):**

1. An internet client sends an HTTP(S) request.
2. In kernel mode, the HTTP Protocol Stack (`HTTP.SYS`) receives the request first. This is the lowest-level, fastest listener.
3. The Windows Activation Service (WAS) and the WWW Service, running inside `svchost.exe`, read the site configuration (`applicationHost.config`) and decide which worker process should handle the request.
4. In user mode, a worker process (`W3WP.exe`) actually processes the request through the Web Server Core pipeline: a. Begin Request Processing b. Authentication c. Authorization d. Cache Resolution e. Handler Mapping f. Handler Pre-execution g. Release State h. Update Cache i. Update Log j. End Request Processing
5. Native modules (for example Anonymous Authentication, IIS Certificate Mapping, Static File handling, Default Document, HTTP Cache, HTTP Errors, HTTP Logging) and the managed engine (for managed modules such as Forms Authentication) run inside this pipeline, inside an application pool and app domain.
6. External applications may be called for dynamic content, similar to Apache's application server step.
7. The response travels back out through `W3WP.exe`, `svchost.exe`, and `HTTP.SYS` to the client.

**Key terms:**

- **HTTP.SYS:** the kernel-mode driver that listens for HTTP traffic on Windows; it is shared by all websites on the server.
- **WAS (Windows Activation Service):** reads configuration and starts or stops application pools and worker processes as needed.
- **W3WP.exe:** the actual worker process where a website's code runs in user mode.

- **Application pool:** an isolation boundary; sites in different application pools do not share a crash or a memory space.
- **Native vs managed modules:** native modules are built into IIS itself (anonymous auth, static file, caching, logging); managed modules run inside the .NET-based managed engine (such as Forms Authentication).

**Real-world use:** A hosting company puts ten customer websites on one IIS server, each in its own application pool. If one customer's site crashes its worker process due to a bug, the other nine keep running because they run in separate `W3WP.exe` processes.

**Tools and commands used here:** None directly; this architecture explains why IIS vulnerabilities (next section) often target either the kernel-mode stack or a specific native/managed module.

**Countermeasures and defense:** Keep sites in separate application pools; disable unused native modules (for example remove Anonymous Authentication if not needed); patch `HTTP.SYS` quickly since a kernel-mode bug can affect the whole server, not just one site.

> **EXAM TIP:**If a question mentions "kernel mode" and a web server, the answer is IIS (`HTTP.SYS`). Apache and Nginx do not have this kernel/user-mode split in the CEH material.

**EXAM FOCUS:** Expect a sequencing question on the Web Server Core pipeline (which step comes before Authorization, etc.) or a "which component runs in kernel mode" question (`HTTP.SYS` only).

**Memory trick:** "Kernel first, then the Crew": `HTTP.SYS` (kernel) greets every request first; the "Crew" (WAS, WWW Service, `W3WP.exe`) in user mode does the actual work afterward.

## 3.6 IIS Vulnerabilities

**What it is (simple words):** A list of real, named weaknesses in IIS itself and in third-party software that runs on top of IIS.

**Vulnerability list (from the notes):**

- **Telerik Report Server trust boundary violation:** weak privilege separation lets an unauthenticated user reach restricted functions.
- **Authentication bypass:** a flaw in the authentication process lets an attacker reach restricted functions and run arbitrary code.
- **SiteMinder Web Agent CRLF/XSS:** misconfiguration lets an attacker run arbitrary JavaScript in a victim's browser.
- **C-CURE 9000 password exposure:** improper logging/handling of sensitive data on the IIS-hosted server exposes passwords to administrators.

- **Aquaforest TIFF Server path access:** default configuration allows directory enumeration/traversal, possibly bypassing authentication.
- **Windows IIS elevation of privilege:** improper handling of user requests lets an attacker gain unauthorized access and control.
- **Hitachi JP1/PM file permissions:** incorrect default permissions let an attacker manipulate files and directories.
- **TYPO3 XSS via PATH_INFO:** unfiltered use of the `PATH_INFO` server variable lets an attacker inject HTML into uncached pages.
- **MailEnable file path handling:** authenticated mail users can place files with unsanitized content into public folders the IIS user can access.
- **XSS in PasswordManager.dll:** unsafe handling of user input in `/isapi/PasswordManager.dll` lets an attacker inject scripts and leak data.

**How it works (general pattern):** Most of these are either (a) an authentication or authorization flaw that lets an attacker skip a check, (b) an input validation flaw (CRLF, XSS, PATH_INFO) that lets an attacker inject data, or (c) a file permission flaw that exposes data or lets files be planted.

**Real-world use:** A company hosts C-CURE 9000 (a physical access control system) on an IIS web front end. Because of improper logging, administrator passwords for the access control system end up readable in logs that lower-privileged admins can also see, letting one of them escalate access to the physical security system.

**Tools and commands used here:** The notes point to `https://` (a cut-off URL, likely a CVE database or vendor advisory page; [UNCLEAR on page 4]).

**Countermeasures and defense:** Patch IIS and any third-party product hosted on it; validate and encode all variables used to build HTML or headers (`PATH_INFO`, custom ISAPI handlers); restrict file write permissions for any feature that lets authenticated users upload files; avoid logging secrets like passwords in plain text.

> **TRAP:**Several of these vulnerabilities are not bugs in IIS itself, they are bugs in products hosted on IIS (Telerik, SiteMinder, C-CURE, Aquaforest, Hitachi, TYPO3, MailEnable). Only "Windows IIS elevation of privilege" is a flaw in IIS itself. A question may test whether you know the difference between "runs on IIS" and "is IIS."

**EXAM FOCUS:** Keywords "PATH_INFO" -> TYPO3 XSS pattern. Keywords "ISAPI" and "PasswordManager.dll" -> that specific XSS. Keywords "default configuration", "directory enumeration" -> Aquaforest-style path access.

**Memory trick:** "Most IIS vulnerabilities are really guest vulnerabilities" -- the house (IIS) is fine, but the tenants (hosted apps) leave the doors open.

## 3.7 Nginx Web Server Architecture

**What it is (simple words):** Nginx (pronounced "engine-x") is a fast web server that can also work as a reverse proxy and a load balancer. It uses one master process that manages several worker processes.

**Analogy:** Nginx is like a restaurant with one manager (master process) and several waiters (worker processes). Each waiter can serve many tables at once without waiting idle for one table to finish eating (non-blocking I/O), instead of the old style where one waiter is stuck with exactly one table until it is done.

**How it works (numbered steps):**

1. Clients send requests to Nginx.
2. The master process starts and supervises multiple worker processes; it does not handle client traffic itself.
3. Each worker process uses non-blocking I/O to manage many connections at once inside a single-threaded loop, so one worker can juggle thousands of connections without creating thousands of threads.
4. Nginx can serve the request directly from its cache (loader, cache manager, memcache, proxy cache) if the content was cached before.
5. If not cached, Nginx forwards the request to a backend: a web server, an application server, a memcached server, or another backend, then caches the result for next time if caching is enabled.
6. Nginx also performs load balancing (spreading requests across several backend servers), SSL/TLS termination (decrypting HTTPS so backend servers do not have to), and detailed logging.

**Key terms:**

- **Master process:** supervises worker processes; handles configuration reload and worker restarts.
- **Worker process:** does the real work of accepting and answering connections.
- **Non-blocking I/O:** a way of handling many network connections without a worker getting stuck waiting on any single one.
- **Reverse proxy:** a server that sits in front of backend servers and forwards client requests to them, hiding the backend from the client.
- **Load balancer:** spreads incoming requests across multiple backend servers so no single server is overwhelmed.

**Real-world use:** A streaming video company puts Nginx in front of ten application servers. Nginx terminates HTTPS, caches thumbnail images so they do not hit the backend every time, and spreads video requests evenly across the ten servers.

**Tools and commands used here:** None directly; feeds into the Nginx vulnerability and misconfiguration sections.

**Countermeasures and defense:** Keep the worker process count tuned to the CPU core count; isolate cache storage with correct permissions; review reverse proxy and load balancer configuration carefully since Nginx misconfigurations are a major theme later in this module.

> **EXAM TIP:**"Master/worker" and "non-blocking, single-threaded loop" are the two phrases that identify Nginx architecture questions. IIS is "kernel mode / user mode". Apache is "modular".

**EXAM FOCUS:** A scenario describing a server handling many thousands of idle-looking connections efficiently, or acting as a reverse proxy and load balancer at the same time, points to Nginx.

**Memory trick:** "One Manager, Many Waiters, No Blocking" for Nginx's master/worker, non-blocking design.

## 3.8 Nginx Vulnerabilities

**What it is (simple words):** A list of real, named weaknesses in Nginx itself and in Nginx-adjacent tools (Nginx-UI, Nginx Management Suite, nginxWebUI, anything-llm).

**Vulnerability list (from the notes):**

- **NULL pointer dereference in HTTP/3:** lets attackers trigger DoS or potentially remote code execution.
- **SSRF in anything-llm:** lets attackers port-scan, reach non-public apps, and delete files.
- **RCE in Nginx-UI:** exposed configuration settings allow remote code execution or privilege escalation.
- **Improper certificate validation in Nginx-UI:** the Import Certificate feature allows arbitrary file writes.
- **SQL injection:** improper neutralization of SQL elements allows unauthorized access or data breaches.
- **Unauthenticated private key access:** Nginx does not support `.htaccess` files, which can expose private keys if misconfigured.
- **Excessive memory/CPU in HTTP/2:** improper memory handling with HTTP/2 requests lets attackers flood and disrupt the server.
- **OS command injection in nginxWebUI:** improper handling of file arguments in the upload feature allows remote command execution.
- **Default file permissions in Nginx Management Suite:** default permissions let attackers modify sensitive files.

**How it works (general pattern):** Many of the newer entries are not about core Nginx code, they are about management add-ons (Nginx-UI, Nginx Management Suite, nginxWebUI) that expose configuration or file upload features to the network.

**Real-world use:** An admin installs Nginx-UI, a web dashboard for managing Nginx, and leaves its configuration page reachable from the internet without authentication. An attacker finds the exposed settings page and uses it to run commands on the server.

**Tools and commands used here:** https://cve.mitre.org (referenced in the notes as the place to look these up).

**Countermeasures and defense:** Never expose management dashboards (Nginx-UI, Nginx Management Suite) directly to the internet; put them behind a VPN or an authenticated reverse proxy; patch Nginx and all management add-ons; set correct file permissions on configuration and certificate files.

> **TRAP:**"Nginx does not support .htaccess files" sounds like a security feature (and generally it is, since it avoids a whole class of Apache .htaccess override attacks), but the notes frame the related private-key exposure as a vulnerability pattern tied to this fact; read the exact wording of a question carefully, since the absence of .htaccess support is not itself the flaw, misconfiguration around it is.

**EXAM FOCUS:** Keywords "Nginx-UI" or "nginxWebUI" or "Nginx Management Suite" point to a management add-on vulnerability, not a flaw in Nginx's core request-handling code. Keywords "HTTP/3 NULL pointer" or "HTTP/2... memory" point to protocol-handling bugs in Nginx core.

**Memory trick:** "The dashboards are the weak door" -- most of Nginx's listed modern CVEs are in the add-on dashboards used to manage it, not in Nginx's own request-handling engine.

## 3.9 DNS Server Hijacking

**What it is (simple words):** The attacker breaks into the DNS server that answers "what is the IP address of this website" and changes its settings, so visitors asking for the real website are instead sent to the attacker's fake server.

**Analogy:** Imagine a phone book that everyone trusts to look up a business's phone number. If someone secretly changes the listed number to their own phone, every customer who "correctly" looks up the business accidentally calls the attacker instead.

**How it works (numbered steps):**

1. The attacker compromises the DNS server (the target) and changes its DNS settings.
2. A user wants to reach the legitimate site and sends a lookup request to the DNS server for the domain name.
3. The DNS server checks its (now tampered) DNS mapping for that domain.
4. The DNS server redirects the user's request to the attacker's malicious server instead of the legitimate site.

5. The user, believing they reached the real site, interacts with the fake site; the attacker can now steal credentials or serve malware.

**Key terms:**

- **DNS mapping:** the record that links a domain name to an IP address.
- **Fake site:** the attacker-controlled server that imitates the real one once DNS points there.

**Real-world use:** An attacker gains admin access to a small ISP's DNS server through a weak password. He changes the A record for `mybank.com` to point to his own server, which hosts a copy of the bank's login page, and harvests every customer's credentials who logs in during the outage before it is detected.

**Tools and commands used here:** None named on this slide specifically (DNS enumeration tools appear in Module 02; defense tools appear later in section 3.27 of this module).

**AI angle:** ADDED CONTEXT: AI-written phishing pages that closely clone a bank's real login page make a DNS-hijacked fake site far more convincing, since the visual copy quality improves and the attacker no longer needs design skill.

**Countermeasures and defense:** See section 3.27 (dedicated DNS hijacking countermeasures slide): registrar locking, safeguarding registrant accounts, DNS monitoring, incident response planning.

> **EXAM TIP:**DNS server hijacking changes settings on the real DNS server. Do not confuse it with DNS cache poisoning (injecting a false record into a resolver's cache without compromising the authoritative server) or with a simple phishing link (no DNS change at all); this module's slide specifically shows the DNS server itself being compromised.

**EXAM FOCUS:** Keywords "compromises the DNS server and changes the settings" -> DNS server hijacking, not cache poisoning. If the question says "spoofed DNS response without touching the server", that is cache poisoning (a different, related attack covered in Module 08 Sniffing).

**Memory trick:** "Hijack the phone book itself, not just one call."

## 3.10 Directory Traversal Attacks

**What it is (simple words):** The attacker uses `../` (dot-dot-slash) sequences in a URL to step out of the website's allowed folder and reach files elsewhere on the server, such as system files.

**Analogy:** A hotel guest is only supposed to access their own room. Directory traversal is like repeatedly pressing "go up one floor, then back down a different stairwell" until the guest ends up in the hotel's locked security office instead of their room.

**How it works (numbered steps):**

1. The attacker finds a script or page on the web server that takes a file path or file name as input (for example in the URL).
2. The attacker tries trial and error with `../` sequences, or encoded variants, to move outside the intended web root directory.
3. Example from the notes: `http://server.com/scripts/..%5c..Windows/System32/cmd.exe?/c+dir+c:\` Here `%5c` is the URL-encoded backslash (`\`), used to escape a weak filter that only blocks the plain `../` or `..\` text.
4. If the server runs the requested executable (`cmd.exe`) with attacker-supplied arguments (`/c+dir+c:\`), the attacker gets command output back, here a directory listing of `C:\`.
5. The sample output shows a `dir /a:h` style listing of the Windows root drive, revealing folders like `inetpub` (the default IIS web root location), `ProgramData`, and `System Volume Information`.

**Key terms:**

- **Dot-dot-slash (`../`):** the standard way to move one directory up in a file path; chaining it moves up multiple levels.
- **URL encoding (`%5c`, `%2e%2e%2f`):** representing special characters like `\`, `.`, `/` using percent-codes so they can pass through filters that only check for the plain character.
- **inetpub:** the default root folder for IIS websites on Windows, containing subfolders such as `wwwroot`, `ftproot`, `logs`, `history`.

**Real-world use:** A legacy IIS application lets users download report files by passing a filename in the URL. The developer only blocks the literal string `../`, so an attacker sends the same path using `%5c` and `..` combinations instead, and reaches `cmd.exe` to run commands on the server, as shown directly in the notes' example URL.

**Tools and commands used here:** `cmd.exe` (Windows command interpreter, abused as the payload target); `dir /a:h` (lists directory contents, including hidden files, from the command line).

**Countermeasures and defense:** Canonicalize and validate all file paths server-side before use; reject any path containing `../` or its encoded forms after decoding; run the web server process with the least privilege needed; keep sensitive system binaries like `cmd.exe` unreachable from the web root logically and by file permissions.

> **TRAP:**Filtering only the literal text `../` is not enough, because encoded forms (`%2e%2e%2f`, `%5c`, double encoding) can bypass a naive filter. The exam loves this exact trap: "the developer blocked `../` but the attack still worked" -> encoding bypass.

**EXAM FOCUS:** Keywords "dot-dot-slash", "../", "outside the web root" -> directory traversal (also called path traversal). Expect a question asking you to identify the encoded bypass character for `\` (`%5c`) or `/` (`%2f`).

**Memory trick:** "Dot dot, out the door" for `../` walking the attacker out of the allowed folder.

## 3.11 Web Server Misconfiguration

**What it is (simple words):** Misconfiguration means the server software is installed correctly but set up carelessly, leaving doors open that should be closed: debug info left on, directory browsing turned on, default accounts still active, unnecessary services running.

**Analogy:** A new house with a working lock on the front door, but the side gate was left unlatched, the garage code is still the factory default, and a window is propped open "just for today." Nothing is broken, everything was simply left in an unsafe default state.

**How it works / examples (from the notes):**

1. **Apache `httpd.conf` server-status exposure:**

   ```
   <Location "/server-status">
     SetHandler server-status
     Require host example.com
   </Location>
   ```

   This configuration allows anyone (if the `Require` rule is too loose, or missing) to view the server status page, which shows detailed information about the web server, current hosts, and requests being processed right now.

2. **Nginx `nginx.conf` unsafe variable usage:**

   ```
   location / {
     set $variable $arg_user_input;
     proxy_pass http://backend/$variable;
   }
   ```

   Taking a user-supplied URL argument (`$arg_user_input`) and dropping it straight into the proxy target without validation can let an attacker manipulate where the proxy sends the request.

3. **IIS `web.config` directory browsing:**

```
<system.webServer>
  <directoryBrowse enabled="true" />
</system.webServer>
```

Turning on `directoryBrowse` lets visitors see a full file listing of a folder when no default page exists there, which can expose sensitive files that were never meant to be linked publicly.

**Real-world use:** A developer enables `directoryBrowse` during testing to quickly check uploaded files, forgets to turn it off before going live, and anyone visiting `/uploads/` later sees a full file listing, including a forgotten backup database file.

**Tools and commands used here:** `httpd.conf`, `nginx.conf`, `web.config` (the three configuration files directly named).

**Countermeasures and defense:** Turn off directory browsing by default; restrict `/server-status` and similar diagnostic endpoints to trusted IP ranges only; never place unvalidated user input directly into a proxy target, header, or file path; review all three config file types during a security audit, not just one web server family.

> **EXAM TIP:**Misconfiguration questions often show you a config file snippet and ask what it enables, or what the fix is. Learn to recognize `directoryBrowse enabled="true"` (IIS directory listing) and `SetHandler server-status` (Apache status page) by sight.

**EXAM FOCUS:** Keywords "verbose debug/error messages", "anonymous or default users/passwords", "unnecessary services enabled" all point straight to the generic term "web server misconfiguration" as the category, even if the exact example differs from these three.

**Memory trick:** "Left the gate, the code, and the window" for Apache (server-status gate), Nginx (unsafe variable "code" path), IIS (open directory "window").

## 3.12 HTTP Response-Splitting Attack

**What it is (simple words):** The attacker sneaks extra header data into an input field so the web server's single reply gets split into two separate HTTP responses, and the attacker controls what the first one contains.

**Analogy:** You hand a waiter a note to pass to the kitchen that says "one burger" but secretly includes a hidden second instruction after a line break that says "and also hand the customer a fake coupon before the food." The kitchen reads it as two separate instructions instead of one, and the fake coupon goes out under the restaurant's own name.

**How it works (numbered steps, from the notes):**

1. The attack adds header response data into an input field so the server splits its single response into two.

2. The attacker exploits a vulnerable server, such as an Apache HTTP server with improper input validation, by injecting a crafted request header that causes the server to generate two responses instead of one.
3. The attacker controls the content of the first response (which can redirect the victim's browser to a malicious site), while the second, leftover response is simply discarded by the browser.

**Worked example from the notes:**

Server-side code (pseudocode, Java-style):

```
String author = request.getParameter(AUTHOR_PARAM);
Cookie cookie = new Cookie("author", author);
cookie.setMaxAge(cookieExpiration);
response.addCookie(cookie);
```

- Normal input `Jason` produces a normal response:

  ```
  HTTP/1.1 200 OK
  Set-Cookie: author=Jason
  ...
  ```

- Malicious input `JasonTheHacker\r\nHTTP/1.1 200 OK\r\n...` (carriage return, line feed characters injected into the `author` parameter) produces a first, attacker-controlled response:

  ```
  HTTP/1.1 200 OK
  Set-Cookie: author=JasonTheHacker
  HTTP/1.1 200 OK
  ...
  ```

  followed by a second, now-discarded response:

  ```
  HTTP/1.1 200 OK
  ...
  ```

**Key terms:**

- **CRLF (Carriage Return, Line Feed):** the two control characters (`\r\n`, URL-encoded as `%0d%0a`) that mark the end of a line in HTTP. Injecting them into an input field that ends up in a header lets an attacker start a brand-new header or even a brand-new response.
- **Input validation:** checking and cleaning user input before using it, here specifically rejecting CRLF characters in values that will be placed into HTTP headers.

**Real-world use:** A website stores the visitor's display name (from a URL parameter) directly into a `Set-Cookie` header with no filtering. An attacker crafts a link containing CRLF sequences in the name parameter; anyone who clicks it has their browser fed an

attacker-controlled extra HTTP response, which the attacker uses to set a malicious cookie or trigger a reflected XSS payload.

**Tools and commands used here:** None named directly; this is a manual crafted-request technique, often found with a web proxy (such as Burp Suite, introduced in Module 14) during testing.

**AI angle:** ADDED CONTEXT: an AI coding assistant can both introduce this bug (by generating header-building code that does not escape CRLF) and help find it (by reviewing code and flagging any place where user input reaches a header without sanitization). Example prompt for defenders: "Review this code for CRLF injection risk in any HTTP header or cookie value built from user input."

**Countermeasures and defense:** See section 3.26 (dedicated countermeasures slide for this attack): disallow `%0d`/`\r` and `%0a`/`\n` characters in any value heading into a header; keep web server software current; comply with RFC 2616 for HTTP/1.1.

> **EXAM TIP:**HTTP response splitting is a root cause, not a final goal. It is used as a stepping stone into reflected XSS, cache poisoning, or sending the victim's browser to a malicious page. If a question's answer options include both "HTTP response splitting" and "cross-site scripting" for the same scenario, check whether the root cause is CRLF header injection (response splitting) feeding into the script injection (XSS), and pick based on exactly what the question asks for (the mechanism vs. the end result).

**EXAM FOCUS:** Keywords "CRLF", "%0d%0a", "two responses from one request" -> HTTP response splitting.

**Memory trick:** "One request in, two replies out" for response splitting's defining trait.

## 3.13 Web Cache Poisoning Attack

**What it is (simple words):** The attacker tricks a shared web cache (a server that stores copies of pages to serve them faster) into storing the attacker's fake page under the address of a real, trusted page. Every normal visitor who then asks the cache for that trusted address gets the attacker's fake page instead.

**Analogy:** A library keeps a popular book on a "quick pick-up" shelf near the door for fast service, labeled with the book's real title. An attacker sneaks their own fake book onto that shelf under the same label. Every customer in a hurry grabs the "quick pick-up" copy and walks away with the attacker's fake content instead of the real book.

**How it works (numbered steps, from the notes):**

1. The attacker sends a request to remove (flush) the real page from the server's cache.

2. The attacker's crafted request (using CRLF injection, similar to section 3.12) generates two responses from the vulnerable server; the attacker keeps the first, real server response.
3. The attacker requests the same legitimate page URL again to force a new cache entry to be created.
4. This time, the attacker receives the second, crafted response from the earlier request, which contains headers (via a redirect script such as `welcome.php?lang=` using `header("Location: ".$_GET['page'])`) that point the cache entry to the attacker's own page.
5. The web cache is now poisoned: its entry for the real address (`www.certifiedhacker.com`) now actually serves "Attacker's page" to every normal user who requests it, instead of the real page.

**Key terms:**

- **Web cache:** a server that stores copies of frequently requested pages so it can answer faster without asking the real (origin) server every time.
- **Cache key:** the address (URL) under which a cached page is stored and looked up; poisoning means putting the wrong content under the right key.
- **Open redirect:** a script that redirects the browser based on an unchecked URL parameter (shown here as `welcome.php?lang=`), often chained into cache poisoning or phishing attacks.

**Real-world use:** A news site uses a CDN-style cache in front of its servers. An attacker exploits an open redirect script and CRLF injection together to make the cache store a fake "breaking news" page under the homepage's real URL; every visitor hitting the cached homepage for the next few hours sees the attacker's fake content instead of real news.

**Tools and commands used here:** None named as a specific tool; this is shown as a manual, crafted HTTP request technique.

**Countermeasures and defense:** See section 3.26: keep web server software current, run vulnerability scanners regularly, disallow CRLF characters in any input that reaches a header, use a single consistent TCP connection per virtual host at the proxy layer, and correctly implement "maintain request host header" at proxies.

> **TRAP:**Web cache poisoning and DNS server hijacking (section 3.9) both end with "the visitor gets a fake page instead of the real one," but the mechanism is completely different: cache poisoning abuses an intermediate cache server's trust in crafted responses; DNS hijacking abuses control over the DNS server's name-to-IP mapping. Read the scenario for which system was actually manipulated.

**EXAM FOCUS:** Keywords "cached content", "intermediate web cache", "poisoned cache entry" -> web cache poisoning. This often pairs with HTTP response splitting as the delivery mechanism.

**Memory trick:** "Poison the shelf, not the book" -- the real server and real page may be untouched, only the cache's copy is corrupted.

## 3.14 SSH Brute Force Attack

**What it is (simple words):** SSH (Secure Shell) builds an encrypted tunnel between two computers so data can travel safely over an unsafe network. An attacker who cannot read the encrypted traffic can instead try to guess the SSH login password over and over, which is called brute forcing.

**Analogy:** SSH is like an armored courier van: nobody outside can see what is inside while it travels. Brute forcing the login is like trying every possible key on the van's door lock until one works, since the armor itself cannot be cracked open directly.

**How it works (numbered steps, from the notes and general knowledge):**

1. A legitimate user connects to a server (web server, mail server, application server, or file server) through an SSH tunnel over the internet.
2. An attacker targets the SSH server and repeatedly tries username and password combinations (a dictionary list or every possible combination) until one succeeds.
3. Once inside, the attacker has an encrypted, trusted-looking channel into the network, which is also useful for sneaking malware or other exploits past network monitoring without being flagged, since the traffic itself is encrypted.

**Key terms:**

- **SSH tunnel:** an encrypted connection between two hosts used to transport data, including commands and file transfers, securely.
- **Brute force attack:** trying many or all possible password guesses until the correct one is found.

**Real-world use:** An attacker scans the internet for machines with port 22 (SSH) open, then runs an automated brute force tool against each one using a list of common usernames (`root`, `admin`) and common passwords. Any server using a weak or default password is compromised within minutes.

**Tools and commands used here:** None named directly on this slide; in practice, tools like Hydra or Medusa (ADDED CONTEXT, standard CEH password-cracking tools, covered fully in Module 06 System Hacking) are used for SSH brute forcing.

**Countermeasures and defense:** Enforce strong, unique passwords or switch to key-based authentication; limit login attempts and lock out or delay after repeated failures; restrict SSH access to specific trusted IP addresses or a VPN; move SSH off the default port 22 as a minor extra hurdle (not a real fix by itself); monitor and alert on repeated failed logins.

> **EXAM TIP:**SSH is encrypted, but encryption protects data in transit, not the login credentials from being guessed. An exam question may try to trick you into thinking "SSH is secure" rules out brute force; it does not, since the attacker is guessing the password itself, not trying to break the encryption.

**EXAM FOCUS:** Keywords "repeatedly try credentials", "SSH tunnel used to smuggle malware undetected" both point here. The notes explicitly also connect SSH tunnels to covert malware delivery because the traffic is encrypted and harder to inspect.

**Memory trick:** "Armored van, weak key" for SSH brute force: the channel is armored (encrypted), the weak point is the key (password).

## 3.15 HTTP/2 Continuation Flood Attack

**What it is (simple words):** A denial-of-service attack against HTTP/2 servers. The attacker opens a connection, sends a HEADERS frame but deliberately never marks it "finished," then keeps sending endless follow-up CONTINUATION frames, forcing the server to keep allocating memory for a header block that never ends, until the server runs out of resources.

**Analogy:** Someone starts dictating a letter to a secretary but never says "end of letter." They keep adding "...and also..." forever. The secretary, following the rules, must keep holding paper and ink ready for a letter that never finishes, until they run out of supplies.

**How it works (numbered steps, from the notes):**

1. The attacker establishes a TCP connection with the victim Apache server.
2. The attacker sends a HEADERS frame without the `END_HEADERS` flag, signaling "more header data is coming."
3. The server allocates memory and resources to hold that HEADERS frame while it waits for the rest.
4. The attacker floods the server with excessive CONTINUATION frames (the "more header data" the server was waiting for), but it never actually ends.
5. The server keeps allocating memory and resources for every CONTINUATION frame it receives.
6. This leads to resource exhaustion in the server's memory.
7. The server crashes or becomes unresponsive, a denial-of-service condition.

**Key terms:**

- **HTTP/2 HEADERS frame:** the frame type that carries header data for a request or response in HTTP/2.
- **END_HEADERS flag:** a flag that tells the receiver "this is the last piece of header data, stop waiting for more."

- **CONTINUATION frame:** a frame that carries additional header data when a HEADERS frame was too large to fit in one frame, or (in this attack) was deliberately left unfinished.

> **ADDED CONTEXT:**This matches the real CVE-2024-27316 class of vulnerability: "Apache HTTP Server: HTTP/2 DoS by memory exhaustion on endless continuation frames" (also seen as CVE-2024-27316, affecting the Apache HTTP Server's HTTP/2 module via the nghttp2 library, verify exact version range and patched version). The underlying technique is widely known as the "HTTP/2 CONTINUATION Flood" and was shown to affect multiple HTTP/2 implementations, not just Apache (Source: IBM security bulletin, Red Hat CVE page). This is the same vulnerability pattern already listed in section 3.4 under Apache vulnerabilities; here the module gives it its own step-by-step attack diagram.

**Real-world use:** A security researcher tests whether a company's Apache-fronted API is patched against this class of bug by opening a connection, sending a HEADERS frame without `END_HEADERS`, and watching server memory usage; on an unpatched server, memory climbs until the process crashes or the connection handler hangs.

**Tools and commands used here:** None named directly in the module; testing tools for this class of bug are typically custom HTTP/2 clients or security scanners aware of the CONTINUATION flood pattern.

**Countermeasures and defense:** Patch Apache (and any HTTP/2 server) to a version that limits the number and total size of CONTINUATION frames per stream and enforces timeouts on incomplete header blocks; monitor memory usage and connection counts for sudden spikes; apply rate limiting at a front proxy or load balancer in front of the origin server.

> **EXAM TIP:**The giveaway phrase for this attack is "HEADERS frame without `END_HEADERS`" followed by a flood of "CONTINUATION frames." If a question describes any other flood (SYN flood, UDP flood, ICMP flood), that belongs to Module 10 Denial-of-Service, not this HTTP/2-specific attack.

**EXAM FOCUS:** This is a protocol-level DoS against the HTTP/2 layer specifically, not the TCP/IP layer. Expect a "which layer/protocol is being abused" question; the answer is the application-layer HTTP/2 framing mechanism.

**Memory trick:** "The letter that never ends" for the endless CONTINUATION frames that keep the server waiting and allocating memory forever.

## 3.16 Frontjacking Attack

**What it is (simple words):** FrontJacking hijacks the front-end (user-facing) behavior of a web application by manipulating a poorly configured Nginx reverse proxy. It combines three techniques at once: CRLF injection, HTTP request header injection, and

XSS, to make the proxy send a victim's traffic to an attacker-controlled backend instead of the real one.

**Analogy:** A shared office building has one receptionist (the Nginx reverse proxy) who routes every visitor to the right company's office based on the name the visitor says at the desk. If an attacker can coach the receptionist into mishearing the name (inject a fake "Host" header), every visitor asking for "Company A" gets walked into the attacker's fake office instead, which looks just like Company A's.

**How it works (numbered steps, from the notes):**

1. The attacker sends an HTTP request to a vulnerable Nginx reverse proxy, injecting a malicious Host header into it (CRLF injection and header injection together).
2. The vulnerable Nginx reverse proxy processes the malicious Host header as if it were legitimate.
3. The proxy routes the request to an attacker-controlled web server instead of the legitimate application server, because its routing logic trusted the Host header value.
4. The attacker-controlled server returns malicious content.
5. The user's browser shows that malicious content, believing it came from the legitimate site.

**Key terms:**

- **Host header:** the HTTP header that tells a server (or, here, a reverse proxy) which website the client is trying to reach; this is essential for virtual hosting (many sites, one IP address).
- **Shared hosting environment:** multiple customers' websites sitting behind the same reverse proxy, which is exactly the setup FrontJacking targets, since the proxy must decide which customer's backend to use based on the Host header.

> **ADDED CONTEXT:**FrontJacking was first documented and named by security researchers studying shared-hosting reverse proxy setups; it is described as combining CRLF injection, HTTP request header injection, and XSS against a poorly configured Nginx reverse proxy. Source: CYE Security blog on FrontJacking, SecureFlag blog on FrontJacking.

**Real-world use:** A web hosting provider runs one Nginx reverse proxy in front of hundreds of customer sites, routed purely by Host header. An attacker who finds a flaw in the proxy's header parsing injects a crafted Host header pointing to their own server; any shared customer whose routing path intersects the flaw can have their visitors silently redirected to the attacker's look-alike page.

**Tools and commands used here:** None named directly; testing this class of bug typically uses a manual or scripted HTTP client capable of sending raw, malformed headers (similar to tools used for HTTP request smuggling research).

**Countermeasures and defense:** Validate and strictly whitelist accepted Host header values at the reverse proxy; never trust a client-supplied Host header for routing decisions without validation; keep Nginx and reverse proxy configuration reviewed and current; isolate shared-hosting customers so a routing flaw cannot cross between them.

> **TRAP:**FrontJacking sounds like "front-end hijacking" of a single page's JavaScript, but it is really a server-side routing/header trust problem at the reverse proxy; the "front" in the name refers to the front-end reverse proxy layer, not client-side JavaScript alone.

**EXAM FOCUS:** Keywords "Nginx reverse proxy," "shared hosting," "Host header," combined with "CRLF" and "XSS" together -> FrontJacking. If only CRLF and header injection are mentioned without the shared-hosting reverse proxy context, consider plain HTTP response splitting instead.

**Memory trick:** "Fool the front desk, redirect the guest" for FrontJacking's Host-header trickery at the reverse proxy.

## 3.17 Other Web Server Attacks

**What it is (simple words):** A short reference list of five more attack categories that affect web servers but are covered in full detail in other CEH modules. This module only summarizes them here to show how they connect to web servers specifically.

**The five, with their home module (from the notes):**

| Attack | One-line summary | Covered fully in |
|---|---|---|
| Password cracking | Exploits weak/flawed authentication | Module 06 |
| DoS/DDoS attacks | Floods server until it crashes or hangs | Module 10 |
| MITM attacks | Intercepts/alters user-server traffic | Module 11 |
| Phishing attacks | Fake site tricks user into login details | Module 09 |
| Web app attacks | Exploits the application running on top | Module 14 |

**EXAM FOCUS:** This slide is a map, not new content; it is there to help you place a question correctly. If a scenario fits one of these five patterns but the question is clearly from Module 13, the expected answer is usually the generic category name (for example "phishing attack" or "DoS attack") rather than a Module-13-specific attack name.

**Memory trick:** "Passwords, Pile-ups, Man in between, Phish, and Apps" for Password cracking, DoS/DDoS (pile-up of requests), MITM, Phishing, and Web app attacks.

## 3.18 Web Server Attack Methodology

**What it is (simple words):** The ordered game plan an attacker (or a pen-tester) follows once they decide to attack a web server, from first look to stealing credentials.

**The six steps (from the notes):**

1. Information Gathering
2. Web Server Footprinting
3. Website Mirroring
4. Vulnerability Scanning
5. Session Hijacking
6. Web Server Passwords Hacking

**Analogy:** This is the same shape as a burglary plan: scout the neighborhood (information gathering), figure out exactly which house and what kind of locks it has (footprinting), make a full copy of the house's layout to study at home (mirroring), check every door and window for weak points (vulnerability scanning), slip in behind someone already inside (session hijacking), and finally pick the safe's combination (password hacking).

**How it works (numbered steps, with what each step delivers):**

1. **Information Gathering:** collect public, low-risk data about the target (robots.txt content, organization details) to build a map of what exists.
2. **Web Server Footprinting:** actively query the server to learn its exact software and version (banner grabbing with Netcat/Telnet, Shodan searches).
3. **Website Mirroring:** copy the entire website locally so it can be studied offline without repeatedly hitting the live server.
4. **Vulnerability Scanning:** run automated scanners against the now well-understood target to find specific exploitable weaknesses.
5. **Session Hijacking:** take over an authenticated user's session (covered fully in Module 11) to skip needing credentials entirely.
6. **Web Server Passwords Hacking:** crack or guess login credentials directly (covered fully in Module 06) when session hijacking is not available or practical.

**EXAM FOCUS:** This six-step order is a strong candidate for a direct "put these steps in order" or "what comes after footprinting" question. Memorize the exact sequence.

**Memory trick:** "I Found Mirrors, Vulnerable to Session, Password" -> Information gathering, Footprinting, Mirroring, Vulnerability scanning, Session hijacking, Password hacking.

## 3.19 Information Gathering from Robots.txt File

**What it is (simple words):** `robots.txt` is a plain text file that tells search engine crawlers which parts of a website they should not index. Ironically, this makes it a map of exactly which folders the site owner considers sensitive enough to hide.

**Analogy:** Posting a sign on your fence that says "please do not look behind the shed, the garage, or the basement window" tells every curious passerby exactly where to look first.

**How it works (numbered steps):**

1. An attacker requests `robots.txt` directly from the target's URL (for example `http://target.com/robots.txt`); no special access is needed, since it is a public file by design.
2. The attacker reads the `Disallow:` lines to learn the root directory structure and sometimes which content management system (CMS) the site uses, based on familiar folder names.
3. The attacker can also download the file in bulk using a tool such as Wget, instead of opening it manually in a browser.

**Sample content shown in the notes:**

```
User-agent: Googlebot
Disallow: /
User-agent: googlebot-image
Disallow: /
User-agent: googlebot-mobile
Disallow: /
User-agent: MSNbot
Disallow: /
User-agent: Slurp
Disallow: /
User-agent: Teoma
Disallow: /
User-agent: Gigabot
Disallow: /
User-agent: ia_archiver
Disallow: /
User-agent: baiduspider
Disallow: /
User-agent: naverbot
Disallow: /
User-agent: Yeti
Disallow: /
```

**Key terms:**

- **User-agent:** in `robots.txt`, identifies which crawler the following rule applies to (`*` means all crawlers).

- **Disallow:** the directive that lists a path the named crawler should not visit or index.
- **Wget:** a command-line tool for downloading files and web content over HTTP, HTTPS, and FTP from the command line.

**Real-world use:** An attacker pulls `robots.txt` from a target online store and sees `Disallow: /admin-panel-backup/`. That single line reveals the existence and name of a folder nobody should have told the public about, which the attacker now tries to access directly.

**Tools and commands used here:** Wget (download `robots.txt` and other files from the command line).

**Countermeasures and defense:** Do not rely on `robots.txt` to hide anything; it is a voluntary, public instruction for well-behaved crawlers, not an access control. Protect sensitive folders with real authentication and access control instead of only listing them in `robots.txt`.

> **TRAP:** `robots.txt` is not a security control. A question may ask "how can an organization protect a sensitive directory listed in robots.txt" and the wrong-but-tempting answer is "remove it from robots.txt"; the correct answer is to add real authentication/access control, since removing the line does not stop a direct request to the folder, and an attacker who already saw it before removal still knows it exists.

**EXAM FOCUS:** Keywords "Disallow:" and "root directory structure leak" point to robots.txt information gathering. Expect a question that tests whether you know robots.txt is advisory only, not enforced.

**Memory trick:** "The 'do not look here' sign points right at it."

## 3.20 Web Server Footprinting / Banner Grabbing

**What it is (simple words):** Actively connecting to a web server and reading its response headers to learn exactly which software and version it runs, plus other system-level details like OS, database schema hints, and account details if leaked.

**Analogy:** Knocking on someone's door and asking "who makes your home security system, and what version?" The homeowner's polite, automatic answer (the HTTP response header) tells you exactly which lock-picking guide to study.

**How it works with Netcat (numbered steps, from the notes):**

1. Connect to the target on port 80: `nc -v www.moviescope.com 80` then press Enter.
2. Send a bare HTTP request: `GET / HTTP/1.0` then press Enter twice (the blank line ends the HTTP request).
3. Read the response headers that come back.

**Sample Netcat output (from the notes):**

```
# nc -v www.moviescope.com 80
DNS fwd/rev mismatch: www.moviescope.com
  != www.goodshopping.com
www.moviescope.com [10.10.1.19] 80 (http) open
GET / HTTP/1.0

HTTP/1.1 200 OK
Content-Type: text/html
Last-Modified: Wed, 15 Apr 2020 06:15:03 GMT
Accept-Ranges: bytes
ETag: "2a415933ed12d61:0"
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 14 Mar 2024 05:51:26 GMT
```

How to read this output, line by line:

- `DNS fwd/rev mismatch` just means forward and reverse DNS lookups do not match; it does not stop the connection.
- `80 (http) open` confirms port 80 is open and speaking HTTP.
- `Server: Microsoft-IIS/10.0` is the prize: the exact web server software and version in use.
- `X-Powered-By: ASP.NET` confirms the backend framework, useful for picking matching exploits.

**How it works with Telnet (same idea, from the notes):**

1. `telnet www.moviescope.com 80` then press Enter.
2. `GET / HTTP/1.0` then press Enter twice.
3. Telnet shows the same style of response:

```
# telnet www.moviescope.com 80
Trying 10.10.1.19...
Connected to www.moviescope.com.
Escape character is '^]'.
GET / HTTP/1.0

HTTP/1.1 200 OK
Content-Type: text/html
Last-Modified: Wed, 15 Apr 2020 06:15:03 GMT
Accept-Ranges: bytes
ETag: "2a415933ed12d61:0"
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 14 Mar 2024 05:57:02 GMT
```

**Key terms:**

- **Banner grabbing:** reading a service's own self-reported identity (here, the `Server:` HTTP header) to learn exactly what it is.

- `Server:` **header:** the HTTP response header most web servers send by default, naming the software and often the version.

**Real-world use:** A pentester runs the exact Netcat command above against a client's public website, sees `Server: Microsoft-IIS/10.0` and `X-Powered-By: ASP.NET`, and now searches specifically for IIS 10.0 and ASP.NET vulnerabilities rather than wasting time on Apache exploits.

**Tools and commands used here:** Netcat (`nc`), Telnet; both are simple network utilities that can read and write raw data across a TCP connection, which is exactly what sending a manual HTTP request needs.

**Countermeasures and defense:** Suppress or customize the `Server:` and `X-Powered-By:` headers so they do not reveal exact software and version; keep whatever version is running fully patched regardless, since hiding the banner is a minor speed bump, not real protection.

> **EXAM TIP:**Both Netcat and Telnet do the exact same manual banner grab here: connect to port 80, send `GET / HTTP/1.0`, press Enter twice. The tool choice does not change the technique; know that both are valid answers to "which tool can be used for manual banner grabbing over HTTP."

**EXAM FOCUS:** Expect an output-reading question: "given this header output, what web server and version is this?" Learn to spot `Server:` and `X-Powered-By:` instantly.

**Memory trick:** "Ask twice, listen once" -- press Enter twice after the `GET` line to send the request, then just read what comes back.

## 3.21 IIS Information Gathering using Shodan

**What it is (simple words):** Shodan is a search engine that indexes internet-connected devices and servers by their banners (what they say about themselves), instead of indexing web page content like Google does. Attackers use it to find IIS servers matching specific criteria without ever touching the target directly.

**Analogy:** Google is like searching a library's book contents. Shodan is like searching a directory of every door in a city labeled with "I am a Ford, model 2023" or "I am IIS, version 10.0", sorted by country, company, and open ports.

**How it works (Shodan search filters from the notes):**

| Filter | What it finds |
|---|---|
| `ssl:"Company Inc." http.title:"IIS"` | IIS servers with SSL certs issued to "Company Inc." |
| `ssl.cert.subject.CN:"company.in" http.title:"IIS"` | IIS servers whose cert common name (CN) is "company.in" |

| Filter | What it finds |
|---|---|
| `http.title:"IIS Windows Server" country:"US"` | IIS Windows Server banners located in the US |
| `http.title:"IIS7" port:80` | IIS7 servers running specifically on port 80 |

**Key terms:**

- **Banner (in Shodan's sense):** the service response Shodan recorded when it last scanned that IP and port, including headers, certificate fields, and titles.
- **ssl.cert.subject.CN:** the Common Name field of an SSL certificate's subject, often the domain or company the certificate was issued to.

**Sample result screen (from the notes):** A Shodan search for `ssl:"Company Inc." http.title:"IIS"` returned 90 total results, with top countries United States, Costa Rica, Puerto Rico, and Chile; top ports 443 and 8443; top organizations including Microsoft Corporation and Comcast Cable; and individual results showing full HTTP response headers (`HTTP/1.1 200 OK`, `Server: Microsoft-IIS/10.0`), SSL certificate details (issuer, validity dates, supported TLS versions: TLSv1, TLSv1.1, and so on).

**Real-world use:** An attacker targeting a specific company first confirms it uses IIS, then runs `ssl.cert.subject.CN:"companyname.com" http.title:"IIS"` on Shodan to find every public-facing IIS server tied to that company's certificates across all its data centers and branch offices, without sending a single probe to the company itself.

**Tools and commands used here:** Shodan (web-based internet device search engine).

**AI angle:** ADDED CONTEXT: AI assistants can help attackers (and defenders doing the same recon on themselves) quickly construct precise Shodan filter syntax from a plain-language goal. Example prompt: "Write a Shodan search filter to find IIS servers in Germany with an SSL certificate issued to 'Example GmbH'."

**Countermeasures and defense:** Avoid unnecessarily exposing banners and certificate details that reveal exact software versions; monitor your own organization's footprint on Shodan regularly as part of external attack surface management; use web application firewalls or reverse proxies to standardize or hide backend banners.

> **EXAM TIP:**Shodan is entirely passive from the target's point of view at query time, since the data was already collected by Shodan's own prior scanning; the attacker's search itself never touches the target.

**EXAM FOCUS:** Keywords "search engine for internet-connected devices", "banners", "ICS", "webcams", "no direct contact at search time" -> Shodan. Expect a filter-syntax matching question (`http.title:`, `ssl.cert.subject.CN:`, `country:`, `port:`).

**Memory trick:** "Shodan reads name tags, Google reads books."

## 3.22 Abusing Apache mod_userdir to Enumerate User Accounts

**What it is (simple words):** Apache's `mod_userdir` module lets each system user publish a personal web page at a URL like `/~username/`. If this feature is enabled, an attacker can test many usernames against the server; a different response for a real username versus a fake one reveals which accounts actually exist.

**Analogy:** A mailroom with a wall of named mail slots. Even if you cannot open any slot, you can tell which names are valid tenants just by whether a slot exists under that name or not.

**How it works (numbered steps, from the notes):**

1. **Perform an initial scan to enumerate valid users:**

   ```
   nmap -p80 --script http-userdir-enum <target>
   ```

   This scans port 80 using the `http-userdir-enum` script, which checks usernames from Nmap's default word list against the `/~username/` pattern.

2. **Perform a customized scan with your own username list:**

   ```
   nmap -p80 --script http-userdir-enum \
     --script-args userdir.users=<wordlist.txt> <target>
   ```

   This tells the script to test usernames from a specific `.txt` file instead of the built-in default list.

3. **Bypass basic detection with a custom User-Agent:**

   ```
   nmap -p80 --script http-brute \
     --script-args http.useragent="<User_Agent>" <target>
   ```

   This changes the HTTP `User-Agent` string Nmap sends, which can help avoid simple detection rules that block the default Nmap signature.

**Flag table:**

| Flag / argument | Meaning | Example |
|---|---|---|
| `-p80` | Only scan port 80 | `-p80` |
| `--script http-userdir-enum` | Run the user directory enumeration NSE script | as shown above |
| `--script-args userdir.users=<file>` | Use a custom wordlist of usernames | `userdir.users=names.txt` |
| `--script-args http.useragent="<value>"` | Set a custom HTTP User-Agent header | `http.useragent="Mozilla/5.0"` |

**Sample output to expect (ADDED CONTEXT, typical NSE-style result):**

```
PORT   STATE SERVICE
80/tcp open  http
| http-userdir-enum:
|   Potential Users:
|     root (403)
|     admin (200)
|_    test (404)
```

How to read it: a `200` or `403` response for `/~admin/` means the `admin` user directory exists on the server (a valid account), while a `404` for `/~test/` means no such user directory exists.

**Common mistakes:** Forgetting `-p80` and scanning all 1000 default ports wastes time; using the default word list against a custom-named internal account naming scheme will miss real users, hence the customized scan with your own list.

**Real-world use:** A pentester targeting a university's Apache server runs `http-userdir-enum` with a wordlist built from staff names found on the university's public directory page, confirming which staff members have active system accounts before attempting password guessing or targeted phishing against exactly those confirmed usernames.

**Tools and commands used here:** Nmap with the `http-userdir-enum` and `http-brute` NSE (Nmap Scripting Engine) scripts.

**Countermeasures and defense:** Disable `mod_userdir` if it is not actually needed; if it must stay enabled, ensure the response for valid and invalid usernames looks identical (same status code and page) so enumeration fails; monitor for repeated requests to many different `/~username/` paths in a short time.

> **EXAM TIP:**This is a direct, exam-friendly NSE script and flag-recall question. Memorize the exact script name (`http-userdir-enum`) and the argument name (`userdir.users`).

**EXAM FOCUS:** Expect "which flag supplies a custom wordlist?" (`--script-args userdir.users=<file>`) and "which script enumerates usernames via mod_userdir?" (`http-userdir-enum`).

**Memory trick:** "User Dir, Under Door" for `mod_userdir` exposing usernames through the `/~username/` "door".

## 3.23 Directory Brute Forcing

**What it is (simple words):** Testing many possible folder and file names against a web server to find pages and directories that are not linked anywhere publicly, and checking whether the server lists their contents.

**Analogy:** Trying every possible room number in a hotel hallway to see which doors are unlocked and which ones have a peephole showing the room's contents (directory listing).

**How it works (numbered steps, from the notes):**

1. When a web server receives a request for a directory, it can respond in one of three ways: a. Returns a default resource inside the directory (like an `index.html`). b. Returns an error (like `404 Not Found`). c. Returns a listing of the directory's contents.
2. Directory listings can be dangerous when there are improper access controls or unintentional access to the web root of the server.
3. After discovering a directory exists on the web server, the attacker requests that same directory path directly to see if it returns a directory listing.
4. The attacker can also try to exploit vulnerable web server software that is known to expose directory listings even when it should not.
5. Tools such as Dirhunt automate this entire search-and-analyze process across a whole site.

**Sample Dirhunt output (from the notes):**

```
$ dirhunt http://www.moviescope.com
Welcome to Dirhunt v1.0.0 using Python 3.11.2
[200] http://www.moviescope.com/
      (HTML document)
[404] http://www.moviescope.com/css/images/
      (Not Found)
[404] http://www.moviescope.com/js/js/
      (Not Found)
[404] http://www.moviescope.com/.well-known/
      security.txt (Not Found)
[404] http://www.moviescope.com:80/2008/
      (Not Found)
```

How to read it: the `[200]` on the homepage confirms it exists and is an HTML document; the repeated `[404]` lines are Dirhunt's probes into guessed or linked paths that do not exist on this particular server, each one ruled out one by one.

**Key terms:**

- **Directory listing:** a page the server auto-generates showing every file in a folder, when no default page (like `index.html`) exists there and listing is enabled.
- **Dirhunt:** described in the notes as a crawler optimized to "search and analyze directories," notable for finding interesting content by following links and page assets rather than relying only on a brute-force wordlist (ADDED CONTEXT, verify exact behavior against the tool's own documentation: Nekmo/dirhunt on GitHub).

**Real-world use:** A pentester runs Dirhunt against a target site and discovers a `/backup/` folder that returns a full directory listing instead of a 404, revealing an old database export file nobody remembered to remove.

**Tools and commands used here:** `dirhunt <url>` (Dirhunt tool, run directly against a target URL).

**Common mistakes:** Assuming a `404` always means "nothing is there"; some servers return a custom `404`-styled page even for real, protected resources, so combining directory brute forcing with manual checks is safer than trusting status codes alone.

**Countermeasures and defense:** Disable directory listing by default on all web servers; apply correct access control to every directory, not just the ones linked from the site's menus; remove unused or forgotten folders (backups, old versions) entirely rather than leaving them reachable.

> **EXAM TIP:**Directory brute forcing and directory traversal (section 3.10) sound similar but are different: traversal walks outside the web root using `../`; brute forcing guesses the names of directories and files inside or around the web root using a wordlist or crawler, without needing to escape it at all.

**EXAM FOCUS:** Keywords "directory listing enabled", "crawler", "finds directories without a wordlist" -> Dirhunt specifically, since the notes highlight it analyzes links rather than only guessing names.

**Memory trick:** "Knock on every door, note which ones answer and which ones show you the whole room."

## 3.24 Vulnerability Scanning

**What it is (simple words):** Running automated tools against a web server to systematically find known weaknesses, misconfigurations, and outdated software, instead of manually checking one thing at a time.

**Analogy:** A home inspector walking through a house with a checklist and sensors, testing every outlet, smoke detector, and window lock in turn, rather than the owner randomly checking one thing they happen to think of.

**How it works (numbered steps, from the notes):**

1. Implement a vulnerability scan to identify weaknesses in the network and determine whether the system can be exploited.
2. Use vulnerability scanners such as Acunetix Web Vulnerability Scanner or Fortify WebInspect to find hosts, services, and vulnerabilities.
3. Sniff network traffic to find active systems, network services, applications, and vulnerabilities present on the network.

4. Test the web server infrastructure for misconfigurations, outdated content, and vulnerabilities using tools like Acunetix.

**Sample Acunetix-style result (from the notes, against `http://testphp.vulnweb.com/`):**

| Severity | Vulnerability | Parameter | Status |
|---|---|---|---|
| High | Cross-site scripting | (various) | Open |
| Medium | nginx SPDY heap buffer overflow | n/a | Open |
| Medium | PHP `allow_url_fopen` enabled | n/a | Open |
| High | SQL injection | login | Open |
| Low | HTML form without CSRF protection | (empty) | Open |
| Medium | Insecure crossdomain.xml file | n/a | Open |
| Low | JetBrains .idea project directory | n/a | Open |

Detail shown for the top finding (Cross-site scripting): the URI was set to `1<ScRiPt>pfta(9580)</ScRiPt>`, and the input was reflected inside a text element on the page, confirming the page echoed unescaped user input straight back into the HTML.

**Key terms:**

- **Vulnerability scanner:** software that automatically probes a target for known issues and reports them, usually with a severity rating.
- **False positive:** a reported vulnerability that is not actually exploitable; part of the analyst's job is confirming real findings.

**Other vulnerability scanning tools named in the notes:**

| Tool | Link |
|---|---|
| Acunetix Web Vulnerability Scanner | https://www.acunetix.com |
| OpenText Fortify WebInspect | https://www.opentext.com |
| Tenable.io | https://www.tenable.com |
| ImmuniWeb | https://www.immuniweb.com |
| Invicti | https://www.invicti.com |

**Real-world use:** A pentester runs Acunetix against a client's test site and gets a list of findings ranked by severity, starting with a confirmed SQL injection in the login

parameter; this becomes the first item in the engagement report because it is the highest-impact, confirmed-open finding.

**Tools and commands used here:** Acunetix Web Vulnerability Scanner (primary tool demonstrated), with WebInspect, Tenable.io, ImmuniWeb, and Invicti listed as alternatives.

**AI angle:** ADDED CONTEXT: many modern vulnerability scanners use machine learning to reduce false positives and to prioritize findings; some also offer AI-generated plain-language remediation guidance for each finding. A defender can prompt an AI assistant with a scanner's raw finding to get a clearer explanation and a suggested code fix.

**Countermeasures and defense:** Run vulnerability scans on a regular schedule, not just once; patch confirmed findings starting with the highest severity; re-scan after fixes to confirm closure.

> **EXAM TIP:**"Which tool would find hosts, services, and vulnerabilities on a web server automatically?" is almost always answered by naming a vulnerability scanner (Acunetix, WebInspect, Tenable.io), not a manual technique like banner grabbing.

**EXAM FOCUS:** Expect a "which tool" matching question across the five named scanners; they are functionally similar for exam purposes (automated web vulnerability scanning), so the exact product named in the question usually just needs to be recognized as "a vulnerability scanner," unless the question tests a specific product feature.

**Memory trick:** "AFTII": Acunetix, Fortify WebInspect, Tenable.io, ImmuniWeb, Invicti -- five scanners, one job.

## 3.25 Nginx Vulnerability Scanning using Nginxpwner

**What it is (simple words):** Nginxpwner is a Python tool built specifically to check an Nginx server for common misconfigurations and known vulnerabilities, rather than being a general-purpose scanner.

**Analogy:** A specialist mechanic who only works on one car brand knows every weak part of that brand by heart, faster than a general mechanic would.

**How it works (numbered steps, from the notes):**

1. Create a temporary file to store a list of potential URL paths to test:

   ```
   nano /tmp/pathlist
   ```

2. Execute the Nginxpwner script against the target, pointing it at the path list file:

```
python3 nginxpwner.py <target_URL> /tmp/pathlist
```

3. Analyze the output Nginxpwner provides to identify potential vulnerabilities and misconfigurations on the target Nginx server.

**Sample output (from the notes):**

```
$ python3 nginxpwner.py http://target/ /tmp/pathlist
NGINXPWNER
A common vulnerability scanner for Nginx
Author: astark0de
NGINX out of date, version is: 1.19.0
and last version is: 1.23.1
Current exploits will be printed now:
Exploits: No Results
Shellcodes: No Results
```

How to read it: the tool first fingerprints the running Nginx version (`1.19.0`) and compares it against the latest known version (`1.23.1`) to flag it as out of date, then separately checks for any matching public exploits or shellcodes for that version.

**Key terms:**

- **Nginxpwner:** per the notes and its GitHub description, "a simple tool to look for common Nginx misconfigurations and vulnerabilities" (Source: stark0de/nginxpwner on GitHub).
- **Gixy:** referenced in the notes as a follow-up tool (https://github.com/yandex/gixy) for finding Nginx configuration misconfigurations if Nginxpwner reveals the `nginx.conf` file itself is exposed, which usually happens because of a missing `root` directive.

**Real-world use:** A pentester finds a target running an old, unpatched Nginx version using Nginxpwner, confirms there are no packaged exploits for that exact version yet, but flags the outdated version itself as a finding requiring a patch before it becomes exploitable.

**Tools and commands used here:** `nginxpwner.py`, `nano` (to build the path list file), and Gixy as a complementary configuration analyzer.

**Common mistakes:** Forgetting to prepare the `/tmp/pathlist` file before running the script, since the tool expects it as an input argument; assuming "no exploit results" means the server is fully safe, when it may just mean no public exploit has been published yet for that specific version.

**Countermeasures and defense:** Keep Nginx updated to the latest stable release; ensure the `nginx.conf` file always has a `root` directive set correctly so configuration details are not accidentally exposed; run Nginxpwner and Gixy against your own servers periodically as part of routine hardening checks.

> **EXAM TIP:** If a question names a tool as specifically built for Nginx misconfiguration scanning, the answer is Nginxpwner, not a general scanner like Acunetix.

**EXAM FOCUS:** Expect a "which tool is Nginx-specific" matching question, and possibly a question on reading version-comparison output ("current version X, latest version Y -> server is outdated").

**Memory trick:** "Pwns Nginx, nothing else" for Nginxpwner's narrow, Nginx-only focus.

## 3.26 Finding Exploitable Vulnerabilities (Exploit-DB)

**What it is (simple words):** Exploit Database (Exploit-DB) is a public, searchable archive of proof-of-concept and working exploit code for known vulnerabilities. Once an attacker knows the exact software and version of a target, Exploit-DB often has ready-made code to try against it.

**Analogy:** A public library section that stores "how someone already broke into this exact model of lock," organized by lock brand and model number, free for anyone to read and copy.

**How it works (numbered steps, from the notes):**

1. Navigate to the Exploit Database website (https://www.exploit-db.com).
2. Use the search bar to enter keywords related to the targeted web server, such as "Apache," "IIS," or "nginx."
3. Apply filters to narrow down the search results.
4. Review the results to identify relevant vulnerabilities.
5. Click into an exploit entry to see detailed information, including the exploit code, description, and references.
6. Cross-check the identified vulnerability against the exact version and configuration of the target web server to confirm it applies.
7. Download the exploit code if it is relevant, and test it against the target web server.

**Sample listing shown in the notes:** An entry titled "Apache mod_proxy_cluster - Stored XSS," with fields for EDB-ID, CVE, EDB Verified status, Author, and Type (WebApps), and a Python exploit script snippet that imports `requests`, `argparse`, and `BeautifulSoup`, then sends a GET request to the target's cluster manager URL as its first step.

**Key terms:**

- **EDB-ID:** Exploit-DB's own internal identifier for a specific exploit entry.
- **CVE:** Common Vulnerabilities and Exposures identifier, the industry-standard reference number for a specific vulnerability.

- **EDB Verified:** a flag showing whether Exploit-DB's maintainers confirmed the exploit actually works as described.

**Real-world use:** After banner grabbing reveals a target runs an old Apache version with `mod_proxy_cluster` enabled, a pentester searches Exploit-DB for "Apache mod_proxy_cluster," finds the exact Stored XSS exploit, confirms the CVE and version match the target, and downloads the script to verify the finding in the lab before reporting it.

**Tools and commands used here:** Exploit Database website (https://www.exploit-db.com); the sample exploit itself uses Python with the `requests` and `BeautifulSoup` libraries.

**Countermeasures and defense:** Patch promptly once a CVE affecting your exact software and version is public, since Exploit-DB makes working attack code freely available almost as fast as the CVE itself becomes known; track your software inventory and versions so you know quickly whether a new Exploit-DB entry applies to you.

> **EXAM TIP:**Exploit-DB is a lookup and research step, not a scanner; it does not probe the target itself. The attacker must already know (from footprinting and banner grabbing) what software and version to search for.

**EXAM FOCUS:** Expect "where would an attacker search for existing, working exploit code for a known vulnerability" -> Exploit-DB.

**Memory trick:** "The library of break-in instructions, filed by lock brand" for Exploit-DB's searchable, software-specific exploit archive.

## 3.27 Path Traversal via Misconfigured NGINX Alias

**What it is (simple words):** A specific Nginx misconfiguration bug in how the `alias` directive is written. If a trailing slash is missing where it should be present, an attacker can manipulate the URL to step outside the folder the administrator meant to expose.

**Analogy:** A sign says "Press button for Floor 3 only," but the elevator's wiring was installed one digit short, so pressing certain extra combinations after it actually unlocks Floor 2 and the basement too.

**How it works (numbered steps, from the notes):**

1. The administrator writes a webroot rule in `nginx.conf` using the `alias` directive, for example:

```
location /i {
  alias /data/w/images/;
}
```

2. Because the `location` path (`/i`) lacks a trailing slash while the `alias` path does have one, Nginx's internal path-matching logic can be manipulated by adding extra path segments after `/i` in the URL.
3. An attacker crafts URLs that walk upward out of the intended `/data/w/images/` folder and into other directories or files outside the designated web root.
4. Tools such as Kyubi automate testing many URL variations to find and confirm this exact traversal pattern.

**Sample Kyubi-style test output (from the notes):**

```
URL                          Status
http://localhost/imgs../     [403]
http://localhost/imgs../../  [200]
http://localhost/i///////./  [200]
```

How to read it: `[403]` (Forbidden) on the first simple attempt shows the server rejected that exact pattern, but adding more `../` and `./` combinations in later attempts eventually returns `[200]` (OK), confirming the traversal succeeded and content outside the intended folder was returned.

**Key terms:**

- **`alias` directive:** in Nginx, maps a URL `location` block to a different filesystem path than the URL would normally suggest.
- **Trailing slash mismatch:** when the `location` path and the `alias` path do not both consistently end in `/`, Nginx's path-matching can be tricked by extra path segments the attacker adds.
- **Kyubi:** per the notes and its documentation, a tool "to discover and exploit Nginx alias traversal misconfiguration" that can brute-force the URL path recursively to find hidden files and directories (Source: shiblisec/Kyubi on GitHub).

**Real-world use:** A photo-sharing site uses `alias /data/w/images/;` under `location /i` without a trailing slash on the location path. A pentester using Kyubi confirms that requesting carefully crafted paths under `/i` returns files from folders outside `/data/w/images/`, proving the misconfiguration is exploitable, not just theoretical.

**Tools and commands used here:** Kyubi (automated Nginx alias traversal discovery and exploitation tool); a cloud shell terminal is shown running it in the notes.

**Countermeasures and defense:** Always match trailing slashes consistently between the `location` path and the `alias` path in Nginx configuration (either both end in `/` or neither does, following Nginx's documented rules); prefer the `root` directive over `alias` where possible, since it is less error-prone for this exact mistake; audit Nginx configuration files specifically for every `alias` directive during a security review.

> **EXAM TIP:**This is a config-syntax trap question category: a tiny missing character (one trailing slash) causes a real, exploitable vulnerability. Expect a question showing an `nginx.conf` snippet and asking what is wrong with it.

**EXAM FOCUS:** Keywords "alias directive," "trailing slash," "Nginx path traversal" -> this specific misconfiguration, distinct from the general directory traversal in section 3.10 (which used `../` against a script's own file-handling code, not a web-server-level routing rule).

**Memory trick:** "One missing slash, one open door" for the trailing slash mismatch in the `alias` directive.

## 3.28 Countermeasures: Patches and Updates

**What it is (simple words):** A disciplined process for applying patches and updates safely, instead of either ignoring them or installing them carelessly.

**How it works (numbered steps, from the notes):**

1. Scan for existing vulnerabilities, then patch and update the server software regularly.
2. Before applying any service pack, hotfix, or security patch, read and peer review all relevant documentation first.
3. Apply all updates, regardless of type, on an "as-needed" basis.
4. Test service packs and hotfixes on a representative non-production environment before deploying them to production.
5. Ensure service packs, hotfixes, and security patch levels are consistent across all domain controllers (DCs).
6. Schedule server outages properly, and make sure a complete set of backup tapes and emergency repair disks are available.
7. Keep a back-out plan that lets the system and the organization return to their original state if an update fails.
8. Schedule periodic service-pack upgrades as part of normal operations maintenance, and never fall more than two service packs behind.

**Key terms:**

- **Hotfix:** a small, often urgent patch for a specific issue, usually released outside the normal update schedule.
- **Back-out plan:** a documented way to undo a change and return to the previous working state if something goes wrong.
- **Domain controller (DC):** a Windows server that manages authentication and security policy for a domain; patch levels must stay consistent across all of them to avoid mismatched behavior.

**Real-world use:** Before rolling out a critical Windows Server patch, an IT team first installs it on a staging server identical to production, confirms the website still works correctly, documents the exact rollback steps, schedules a maintenance window, and only then applies it to the live servers, one at a time.

**Countermeasures and defense:** This whole slide is the countermeasure itself; the main risk being prevented is both "staying unpatched too long" and "patching carelessly and breaking production."

> **EXAM TIP:**"Never trail by more than two service packs" is a specific, memorable number that the exam can quote directly; treat it as a fact to recall, not just a general idea.

**EXAM FOCUS:** Expect a "what should be done before applying a patch in production" question; the correct order is read/review documentation, test in a non-production environment, then deploy with a back-out plan ready.

**Memory trick:** "Read, Test, Back out, Repeat" for the patch lifecycle: read documentation, test off-production, keep a back-out plan, and repeat this on a schedule.

## 3.29 Countermeasures: Protocols and Accounts

**What it is (simple words):** Two groups of hardening steps: locking down which network protocols are allowed, and cleaning up which user accounts exist and what they can do.

**Protocols (from the notes):**

1. Block all unnecessary ports, ICMP traffic, and unnecessary protocols such as NetBIOS and SMB.
2. Harden the TCP/IP stack and consistently apply the latest software patches and updates to system software.
3. If insecure protocols such as Telnet, POP3, SMTP, or IMAP are used, take appropriate measures to provide secure authentication and communication, for example by using IPsec policies.
4. If remote access is needed, ensure remote connections are secured properly using tunneling and encryption protocols.
5. Disable WebDAV if it is not used by the application, or keep it secured if it is required.

**Accounts (from the notes):**

1. Remove all unused modules and application extensions.
2. Disable unused default user accounts created during OS installation.
3. When creating a new web root directory, grant appropriate access permissions to anonymous users.

4. Eliminate unnecessary database users and stored procedures, and follow least privilege for the database application, to defend against SQL query poisoning.
5. Use secure web permissions, NTFS permissions, and .NET Framework access control mechanisms, including URL authorization.
6. Slow down brute force and dictionary attacks with strong password policies, and implement audits and alerts for login failures.

**Key terms:**

- **NetBIOS / SMB:** older Windows file-sharing and name-resolution protocols, both historically targeted by many worms and lateral movement techniques; block them on internet-facing servers.
- **WebDAV:** an extension to HTTP that lets users manage files on a web server remotely; useful but often abused if left open and unauthenticated.
- **IPsec:** a suite of protocols that authenticates and encrypts IP packets, usable to secure otherwise-insecure protocols like Telnet.
- **Least privilege:** giving every account, process, or user only the access it actually needs, nothing more.

**Real-world use:** A sysadmin audits a web server and finds SMB still enabled from the original OS install, a default guest account still active, and a database account with full admin rights used just to run read-only reports. All three get fixed: SMB disabled, the guest account removed, and the database account cut down to read-only access on only the tables the report actually needs.

**Countermeasures and defense:** This entire slide is itself the countermeasure list; the underlying risks are protocol-level exposure (NetBIOS/SMB, insecure legacy protocols) and account-level over-privilege (default accounts, over-permissioned database users).

> **EXAM TIP:**"Follow least privilege for the database application to defend against SQL query poisoning" is a direct line connecting an account hardening step to a specific attack category (SQL injection is sometimes called "SQL query poisoning" in older EC-Council material); recognize both names as the same idea.

**EXAM FOCUS:** Expect "which protocol should be blocked on an internet-facing web server" (NetBIOS, SMB, unnecessary ICMP) and "which account practice reduces brute force risk" (strong password policy plus login failure auditing and alerting).

**Memory trick:** "Block the chatty protocols, trim the fat accounts."

## 3.30 Countermeasures: Files and Directories

**What it is (simple words):** A set of file-system-level hardening steps focused on removing unneeded files, hiding sensitive configuration, and keeping logs and application files properly separated and monitored.

**How it works (numbered steps, from the notes):**

1. Eliminate unnecessary files within `.jar` files.
2. Disable the serving of directory listings.
3. Eliminate sensitive configuration information stored within the byte code.
4. Avoid mapping virtual directories between two different servers, or over a network.
5. Monitor and check all network service logs, website access logs, database server logs (Microsoft SQL Server, MySQL, Oracle), and OS logs frequently.
6. Eliminate non-web files such as archive files, backup files, text files, and header/include files.
7. Disable serving certain file types by creating a resource mapping.
8. Ensure web application or website files and scripts are stored on a partition or drive separate from the OS, logs, and other system files.

**Key terms:**

- **Virtual directory:** a URL path that maps to a physical folder that may not match the URL's apparent location; mapping it across different servers or over a network adds risk and complexity.
- **Byte code:** compiled code (such as Java `.class` files packed in a `.jar`); if sensitive configuration (passwords, keys) is embedded in it, it can often be decompiled and read back out.
- **Resource mapping:** a web server setting that controls which file extensions the server will serve or execute; disabling serving of certain extensions (like `.bak`, `.old`) prevents accidental exposure.

**Real-world use:** An auditor finds an old `.bak` copy of a config file sitting in the web root with database credentials embedded in a `.jar` file's byte code; both get removed, the file type `.bak` gets blocked from being served at the resource-mapping level, and the application's files get moved onto their own dedicated drive, separate from the OS and log drives.

**Countermeasures and defense:** This slide's eight points are themselves the countermeasures; the underlying risk pattern is "forgotten files quietly becoming reachable," whether that is a leftover backup, a decompiled secret, or an over-permissive virtual directory mapping.

> **EXAM TIP:**"Separate partition for web files, logs, and OS files" is a classic exam fact: it limits how far a single directory traversal or disk-filling attack can spread, since an attacker confined to the web files' partition cannot automatically reach OS files on a different partition.

**EXAM FOCUS:** Expect "why should web application files be on a separate partition from the OS" (limits the blast radius of traversal or disk-based attacks) and "what

should never be embedded in compiled byte code" (sensitive configuration like passwords or keys).

**Memory trick:** "Different drive, clean shelf, no crumbs left behind" for separate partitions, no directory listings, and no leftover backup/archive files.

## 3.31 How to Defend against Web Server Attacks (Ports, Machine.config, Certificates, Code Access Security)

**What it is (simple words):** A focused checklist covering four specific hardening areas: network ports, the IIS `machine.config` file, server certificates, and code access security settings.

**Ports (from the notes):**

1. Regularly audit the ports on the server to ensure no insecure or unnecessary service is active.
2. Limit inbound traffic to port 80 for HTTP and port 443 for HTTPS (SSL/TLS).
3. Encrypt or restrict intranet traffic.

**Machine.config (from the notes):**

1. Ensure protected resources are mapped to `HttpForbiddenHandler`, and remove unused `HttpModules`.
2. Ensure tracing is disabled (`<trace enable="false"/>`) and debug compiles are turned off.

**Server certificates (from the notes):**

1. Ensure certificate date ranges are valid and that certificates are used for their intended purpose.
2. Ensure no certificate has been revoked, and that the certificate's public key chains all the way to a trusted root authority.

**Code access security (from the notes):**

1. Implement secure coding practices and restrict code access security policy settings.
2. Configure IIS to reject URLs containing `..` and install new patches and updates.

**Key terms:**

- **HttpForbiddenHandler:** an ASP.NET handler that simply denies all requests to whatever resource it is mapped to, used to block access to protected file types or paths.
- **HttpModules:** pluggable components in the ASP.NET request pipeline; unused ones should be removed to shrink the attack surface, the same idea as disabling unused Apache modules.

- **Tracing / debug compiles:** development features that print extra internal detail; leaving them on in production can leak sensitive information to attackers.
- **Code access security (CAS):** a .NET security model that restricts what code is allowed to do based on its trust level and origin.

**Real-world use:** A security review of an ASP.NET site finds tracing left enabled from development, exposing internal stack traces to any visitor who triggers an error; disabling `<trace enable="false"/>` and turning off debug compiles immediately stops that information leak.

**Countermeasures and defense:** This entire slide is the countermeasure list; port restriction, `machine.config` hardening, certificate validity checks, and code access security together close off several independent leak and bypass paths at once.

> **EXAM TIP:**"Configure IIS to reject URLs with `..`" directly defends against the directory traversal pattern from section 3.10; connect this specific IIS setting to that earlier attack when you see either one in a question.

**EXAM FOCUS:** Expect "which setting disables detailed error/debug output on an ASP.NET site" (`trace enable="false"` and debug compiles off) and "which ports should remain open on a public web server" (80 and 443 only, by default).

**Memory trick:** "Ports closed, traces off, certs valid, dots rejected" for the four focus areas in this slide.

## 3.32 How to Defend against Web Server Attacks (Cont'd) - Shares, Permissions, Script Mappings

**What it is (simple words):** More Windows/IIS-specific hardening steps focused on remote administration, file shares, NTFS permissions, and unnecessary ISAPI filters or script mappings.

**How it works (numbered steps, from the notes):**

1. Apply restricted ACLs (Access Control Lists) and block remote registry administration; secure the SAM (Security Accounts Manager) on stand-alone servers only.
2. Ensure security-related settings are configured appropriately and that access to the metabase file is restricted with hardened NTFS permissions.
3. Remove unnecessary ISAPI filters from the web server.
4. Remove all unnecessary file shares, including the default administration shares, if they are not required.
5. Secure the remaining shares with restricted NTFS permissions.
6. Relocate sites and virtual directories to non-system partitions, and use IIS web permissions to restrict access.

7. Remove all unnecessary IIS script mappings for optional file extensions, to avoid exploitation of bugs in the ISAPI extensions handling those file types.
8. Enable a minimum level of auditing on the web server, and use NTFS permissions to protect the log files themselves.

**Key terms:**

- **ACL (Access Control List):** a list of permissions attached to a resource, defining which accounts can access it and how.
- **SAM (Security Accounts Manager):** the Windows database that stores local user account and password information.
- **Metabase:** the configuration storage file used by older versions of IIS (replaced by `applicationHost.config` in later IIS versions, ADDED CONTEXT, verify version where this transition happened).
- **ISAPI filter:** a module that processes every request to IIS before it reaches the final handler; unnecessary filters widen the attack surface for no benefit.
- **Script mapping:** the IIS setting that connects a file extension (like `.asp` or `.php`) to the engine that should execute it; mapping extensions that are not actually used leaves an unused, exploitable path open.

**Real-world use:** An IIS server audit finds the default administrative file shares still enabled, several ISAPI filters left over from an uninstalled product, and script mappings still pointing to a `.php` engine on a server that only serves ASP.NET content; all three findings get removed, shrinking the attack surface without changing any actual site functionality.

**Countermeasures and defense:** This slide's eight points are the countermeasures themselves; the shared theme is removing anything "left over" (shares, filters, script mappings) that is not actively required for the site to function.

> **EXAM TIP:**"Remove unnecessary IIS script mappings for optional file extensions" is one of the most commonly tested IIS hardening facts; remember it specifically protects against bugs in ISAPI extensions tied to file types the site does not even use.

**EXAM FOCUS:** Expect "why remove unused script mappings" (prevents exploitation of unused ISAPI extension bugs) and "where should sites and virtual directories be placed" (on non-system partitions).

**Memory trick:** "If it is not used, remove it: shares, filters, mappings alike."

## 3.33 How to Defend against Web Server Attacks (Cont'd) - General Hardening Practices

**What it is (simple words):** A broader, general-purpose hardening checklist covering the dedicated server principle, physical security, and traffic filtering, continuing numbered from item 8.

**How it works (numbered list, from the notes, continuing the numbering shown on the slide):**

1. Use a dedicated machine as a web server.
2. Create URL mappings to internal servers cautiously.
3. Do not install the IIS server on a domain controller.
4. Use server-side session ID tracking and match connections with timestamps, IP addresses, and similar identifying data.
5. If a database server such as Microsoft SQL Server is used as a backend database, install it on a separate server.
6. Use security tools provided with the web server software, and scanners that automate and simplify securing a web server.
7. Physically protect the web server machine in a secure machine room.
8. Do not connect an IIS server to the internet until it is fully hardened.
9. Do not allow anyone to log in locally to the machine except the administrator.
10. Configure a separate anonymous user account for each application, if multiple web applications use anonymous access.
11. Limit the server's functionality to support only the web technologies actually in use.
12. Screen and filter incoming traffic requests.

**Key terms:**

- **Dedicated machine:** a server used for exactly one role (web server only), so a compromise of one role cannot automatically reach another role's data, directly echoing the "no dedicated server" cause of compromise named back in section 3.2.
- **Domain controller:** should never also be a web server, since a web server compromise could then cascade into full domain-wide authentication compromise.

**Real-world use:** A company that originally ran its public website on the same box as its Active Directory domain controller (to save on hardware) separates the two after a security review, following item 10 directly, because any future web server compromise would otherwise risk the entire domain's authentication system.

**Countermeasures and defense:** This slide is itself the countermeasure list; its central theme is isolation (dedicated roles, separate anonymous accounts per app, physical security) and minimalism (limit functionality to only what is needed).

> **EXAM TIP:**"Do not install IIS on a domain controller" is a very frequently tested, specific, memorable fact; expect it to appear as a direct true/false or best-practice question.

**EXAM FOCUS:** Expect "which practice prevents a web server compromise from becoming a domain-wide compromise" (do not run IIS on a domain controller; use a dedicated machine).

**Memory trick:** "One box, one job, one locked door" for dedicated server roles, limited functionality, and physical security together.

## 3.34 How to Defend against HTTP Response-Splitting and Web Cache Poisoning

**What it is (simple words):** Defense recommendations split across three different roles, since no single team can fix response splitting or cache poisoning alone: server admins, application developers, and proxy server operators.

**How it works (by role, from the notes):**

**Server Admin:**

1. Use the latest web server software.
2. Regularly update and patch the OS and web server.
3. Run a web vulnerability scanner.

**Application Developers:**

1. Restrict web application access to unique IPs.
2. Disallow carriage return (`%0d` or `\r`) and line feed (`%0a` or `\n`) characters in any input that reaches a header.
3. Comply with RFC 2616 specifications for HTTP/1.1.

**Proxy Servers:**

1. Avoid sharing incoming TCP connections among different clients.
2. Use different TCP connections with the proxy for different virtual hosts.
3. Implement "maintain request host header" correctly.

**Key terms:**

- **RFC 2616:** the specification document defining HTTP/1.1, which response-splitting attacks violate by injecting characters HTTP headers were never meant to contain.
- **Virtual host:** a single physical server hosting multiple distinct websites or domains, each needing correct, unmixed traffic handling at the proxy layer.

**Real-world use:** A company's security team assigns three different fixes to three different teams after an HTTP response splitting incident: the infrastructure team patches and updates the web server software, the developers add input filtering to strip CRLF characters from any value used in a header or cookie, and the network team reconfigures the front proxy so different customer virtual hosts never share the same backend TCP connection.

**Countermeasures and defense:** This slide's three role-based lists are the countermeasures themselves, directly closing the exact mechanisms shown in sections 3.12 and 3.13 (CRLF injection and the cache behaviors it enables).

> **EXAM TIP:**This is one of the few CEH slides that splits countermeasures by job role rather than by technique; expect a question asking "whose responsibility is it to disallow CRLF characters in input" (application developers) versus "whose responsibility is it to avoid sharing TCP connections among clients" (proxy server operators).

**EXAM FOCUS:** Keywords "%0d", "%0a", "CRLF" in a developer-focused answer option; keywords "shared TCP connections", "virtual hosts" in a proxy-focused answer option.

**Memory trick:** "Admin patches, Developer filters, Proxy separates" for the three roles' three different jobs.

## 3.35 How to Defend against DNS Hijacking

**What it is (simple words):** A focused, numbered list specifically for preventing the DNS server hijacking attack covered earlier in section 3.9.

**How it works (numbered steps, from the notes):**

1. Choose an ICANN-accredited registrar, and encourage them to set Registrar-Lock on the domain name.
2. Safeguard the registrant's account information.
3. Include DNS hijacking in incident response and business continuity planning.
4. Use DNS monitoring tools/services to monitor the IP address of the DNS server, and set alerts for changes.
5. Avoid downloading audio/video codecs and other downloaders from untrusted websites.
6. Install an antivirus program, and update it regularly.
7. Change the default router password that comes with factory settings.

**Key terms:**

- **ICANN:** the Internet Corporation for Assigned Names and Numbers, the organization that oversees domain name registration policy globally; registrars must be accredited by it to sell domain names.

- **Registrar-Lock:** a setting a domain registrar can apply that blocks unauthorized transfers or changes to a domain's registration without extra verification.
- **Business continuity planning:** the organization's documented plan for continuing operations through a disruptive incident, which should explicitly include DNS hijacking as a scenario to prepare for.

**Real-world use:** A company enables Registrar-Lock on its main domain, enforces multi-factor authentication on its registrar account, subscribes to a DNS monitoring service that alerts the IT team the moment the domain's DNS records change unexpectedly, and had already written a DNS-hijacking response plan before an incident, so the team reacts within minutes instead of hours when an alert fires.

**Countermeasures and defense:** This entire slide is the countermeasure list for DNS hijacking specifically; note that two of the seven items (avoiding untrusted downloaders, antivirus, router password) address how an attacker's malware could first gain the foothold needed to reach and manipulate DNS settings in the first place, not just the DNS server itself.

> **TRAP:**Items 5, 6, and 7 (malware downloads, antivirus, router password) look unrelated to "DNS hijacking" at first glance, but they address the common ways an attacker first gets a foothold (infected downloads, malware, a weak home/office router) that can then be used to change DNS settings; a question may test whether you understand this connection rather than expecting these to be obviously about DNS alone.

**EXAM FOCUS:** Keywords "Registrar-Lock", "ICANN accredited registrar", "DNS monitoring tools" all point directly to this slide's countermeasures for DNS hijacking specifically, distinct from the general web server hardening in sections 3.28 to 3.33.

**Memory trick:** "Lock the registrar, watch the records, guard the router" for the registrar lock, DNS monitoring, and router password items together.

## 3.36 Web Server Security Tools

**What it is (simple words):** A named list of commercial security products used to find and manage web server vulnerabilities on an ongoing basis, beyond the vulnerability scanners already covered in section 3.24.

**Tools (from the notes, with their described purpose):**

- **OpenText Fortify WebInspect:** an automated dynamic application security testing (DAST) solution that discovers configuration issues and identifies and prioritizes security vulnerabilities in running applications (https://www.opentext.com).
- **Acunetix Web Vulnerability Scanner** (https://www.acunetix.com).
- **NetIQ Secure Configuration Manager** (https://www.netiq.com).

- **SAINT Security Suite** (https://www.carson-saint.com).
- **Sophos Intercept X for Server** (https://www.sophos.com).
- **UpGuard** (https://www.upguard.com).

**Key terms:**

- **Dynamic Application Security Testing (DAST):** testing a running application from the outside, the same way an attacker would, rather than reading its source code (which is called static testing, ADDED CONTEXT, covered more in Module 14).
- **Secure Configuration Manager:** a class of tool, like NetIQ's, focused on checking that systems follow secure configuration baselines, closely related to the misconfiguration theme from section 3.11.

**Real-world use:** A company subscribes to Sophos Intercept X for Server for ongoing endpoint and server protection, uses NetIQ Secure Configuration Manager to continuously check its web servers against a hardening baseline, and runs Fortify WebInspect before every major release to catch new vulnerabilities before going live.

**Tools and commands used here:** OpenText Fortify WebInspect, Acunetix, NetIQ Secure Configuration Manager, SAINT Security Suite, Sophos Intercept X for Server, UpGuard.

**Countermeasures and defense:** Using tools from this list regularly is itself a countermeasure, turning the earlier one-time hardening checklists (sections 3.28-3.35) into an ongoing, monitored process.

> **EXAM TIP:**"DAST" is a term worth recognizing by name; WebInspect is the clearest example of a DAST tool named directly in this module.

**EXAM FOCUS:** Expect a simple recall or matching question naming one of these six tools and asking what general category it belongs to (web vulnerability scanner, configuration manager, endpoint protection).

**Memory trick:** "Fortify tests running apps; NetIQ checks config; Sophos guards the endpoint" for three of the six tools' distinct roles.

## 3.37 Web Server Pen Testing Tools

**What it is (simple words):** A named list of tools used specifically during authorized penetration testing engagements against web servers, rather than for everyday defensive scanning.

**Tools (from the notes, with their described purpose):**

- **CORE Impact:** finds vulnerabilities on an organization's web server, and lets a user evaluate the security posture of a web server using present-day cybercrime techniques (https://www.coresecurity.com).
- **Cobalt Strike** (https://www.cobaltstrike.com).
- **Fuxploider** (https://github.com).
- **mitmproxy** (https://mitmproxy.org).

**Key terms:**

- **CORE Impact:** a commercial, full-lifecycle penetration testing platform that can validate exploitability, not just detect theoretical vulnerabilities.
- **Cobalt Strike:** a well-known commercial adversary simulation and red-team platform, frequently referenced across multiple CEH modules for post-exploitation and command-and-control activity (ADDED CONTEXT, covered more broadly in other modules).
- **Fuxploider:** per its project description, a tool focused on automatically finding and exploiting file upload vulnerabilities on web servers (ADDED CONTEXT, verify exact scope against the tool's own documentation).
- **mitmproxy:** an interactive, open-source tool for intercepting, inspecting, modifying, and replaying HTTP(S) traffic between a client and server, useful for manually testing things like HTTP response splitting and cache poisoning from earlier sections.

**Real-world use:** A red team uses CORE Impact to validate that a discovered vulnerability on a client's web server is truly exploitable, then uses mitmproxy to intercept and modify live requests to confirm a cache poisoning finding works exactly as expected, and documents both with proof for the final report.

**Tools and commands used here:** CORE Impact, Cobalt Strike, Fuxploider, mitmproxy.

**AI angle:** ADDED CONTEXT: AI-assisted red-teaming increasingly pairs with tools like mitmproxy, using an AI assistant to help craft or explain intercepted and modified request payloads during manual testing.

**Countermeasures and defense:** The existence of these pen-testing tools is itself a reminder: assume an attacker has access to equally capable (or identical) tools, and defend accordingly using every countermeasure covered earlier in this module.

> **EXAM TIP:**Distinguish vulnerability scanners (section 3.24, find and report issues) from pen-testing tools (this section, actively exploit and validate issues); CORE Impact and Cobalt Strike both emphasize validating exploitability, not just detection.

**EXAM FOCUS:** Expect "which tool intercepts and modifies HTTP traffic for manual testing" -> mitmproxy. "Which tool focuses on file upload vulnerabilities" -> Fuxploider.

**Memory trick:** "Impact proves it, Cobalt simulates it, Fuxploider uploads it, mitmproxy watches it" for the four tools' distinct roles.

### 3.38 Module Summary

**What it is (simple words):** The closing slide restates the module's four major areas and previews that Module 14 (Hacking Web Applications) comes next.

**Summary points (from the notes):**

1. Web server concepts.
2. Various web server threats and attacks in detail.
3. Web server attack methodology in detail, including information gathering, web server footprinting, vulnerability scanning, and web server passwords hacking.
4. Various countermeasures employed to prevent web server hacking attempts by threat actors.
5. Detailed discussion on securing web servers using various security tools.

**EXAM FOCUS:** The module summary itself is rarely tested directly, but it confirms the module's scope boundary: Module 13 ends at the web server layer; Module 14 picks up specifically at the web application layer that runs on top of it. If a question's scenario is clearly about application logic (like a shopping cart bug) rather than the server software itself, it likely belongs to Module 14, not 13.

**Memory trick:** "Concepts, Combat, Course, Cure, Covered" closing out the same four-part memory trick from section 3.1, plus the hand-off to Module 14 next.

## 4. Tool Encyclopedia

| Tool | What it does (simple) | Type/Platform | Phase used | Exam keyword |
|---|---|---|---|---|
| Netcat (nc) | Raw TCP read/ write for manual banner grab | CLI | Footprinting | "Server:" header |
| Telnet | Same manual banner grab job as Netcat | CLI | Footprinting | "GET / HTTP/1.0" |
| Shodan | Searches devices by banner, not page text | Web | Footprinting | "banners", filters |

| Tool | What it does (simple) | Type/Platform | Phase used | Exam keyword |
|---|---|---|---|---|
| Wget | Downloads files/ pages from the CLI | CLI | Info gathering | bulk robots.txt |
| Nmap (NSE) | Port scan plus scripted tasks like enum | CLI | Footprint/enum | http-userdir-enum |
| Dirhunt | Crawls links to find directories | CLI, Python | Dir discovery | "without bruteforce" |
| Acunetix | Automated web vulnerability scanner | GUI, commercial | Vuln scanning | "vulnerability scanner" |
| Fortify WebInspect | Automated DAST, finds and ranks flaws | GUI, commercial | Vuln scanning | "DAST" |
| Tenable.io | Cloud vulnerability management | Web, commercial | Vuln scanning | vuln management |
| ImmuniWeb | Web vuln scanning with reporting | Web, commercial | Vuln scanning | "Generate Report" |
| Invicti | Automated web vulnerability scanner | GUI/web, commercial | Vuln scanning | web vuln scanner |
| Nginxpwner | Scans Nginx misconfigs/ vulnerabilities | CLI, Python | Vuln scanning | "Nginx out of date" |
| Gixy | Analyzes nginx.conf for misconfigs | CLI, Python | Vuln scanning | nginx.conf |
| Exploit-DB | Archive of exploit code by CVE/ EDB-ID | Web | Exploit research | EDB-ID, CVE |
| Kyubi | Exploits Nginx alias traversal bug | CLI | Exploitation | "alias directive" |
| CORE Impact | Finds, validates web server exploits | GUI, commercial | Pen testing | "security posture" |

| Tool | What it does (simple) | Type/Platform | Phase used | Exam keyword |
|---|---|---|---|---|
| Cobalt Strike | Adversary simulation/red team platform | GUI, commercial | Pen testing | red team |
| Fuxploider | Finds/exploits file upload flaws | CLI | Pen testing | file upload flaw |
| mitmproxy | Intercepts/ modifies HTTP(S) traffic | CLI/TUI, open source | Pen testing | intercept traffic |
| NetIQ SCM | Checks secure config baselines | GUI, commercial | Hardening/audit | config baseline |
| SAINT Security Suite | Vuln assessment/ management suite | GUI, commercial | Vuln scanning | security suite |
| Sophos Intercept X for Server | Endpoint/server protection product | Agent/GUI, commercial | Defense | server protection, endpoint |
| UpGuard | External attack surface and risk monitoring | Web-based, commercial | Monitoring/ defense | attack surface monitoring |

> **ADDED CONTEXT:**The exam sometimes places close alternatives next to the tools above. Common pairs to recognize: Burp Suite (manual web proxy, closer to mitmproxy's role but introduced fully in Module 14) vs mitmproxy (open source, scriptable); Nessus/OpenVAS (general network vulnerability scanners) vs Acunetix/ Invicti (web-focused application scanners); Metasploit (general exploitation framework) vs CORE Impact (commercial full-lifecycle pen test platform).

# 5. Command Sheet

I have no lab, so each entry below explains the command fully: purpose, syntax, what each part means, flags, example commands, a realistic sample output, how to read that output, common mistakes, and what the exam usually asks about it.

## 5.1 Netcat manual banner grab

**Purpose:** Open a raw TCP connection to a web server's port 80 and manually send an HTTP request to read the response headers.

**Full syntax:**

```
nc -v <target> 80
```

Then, after the connection opens, type:

```
GET / HTTP/1.0
```

and press Enter twice (an empty line ends the HTTP request).

**Breakdown of each part:**

- `nc` : the Netcat program.
- `-v` : verbose mode, prints connection status messages.
- `<target>` : the hostname or IP address to connect to.
- `80` : the destination TCP port (HTTP).
- `GET / HTTP/1.0` : a bare HTTP/1.0 request for the root page `/`.

**Flag table:**

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| `-v` | Verbose output | Always, to confirm connection status | `nc -v target 80` |
| `-w <secs>` | Set connection timeout in seconds | Slow or unreliable networks | `nc -w 5 target 80` |
| `-n` | Skip DNS resolution, use IP only | Target DNS is broken or you want speed | `nc -n 10.10.1.19 80` |

**Demo commands in a realistic scenario:**

```
nc -v www.moviescope.com 80
```

```
nc -w 5 -v www.moviescope.com 80
```

**Sample output (realistic):**

```
www.moviescope.com [10.10.1.19] 80 (http) open
GET / HTTP/1.0

HTTP/1.1 200 OK
Content-Type: text/html
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 14 Mar 2024 05:51:26 GMT
```

**How to read it, line by line:**

1. `80 (http) open` confirms the port is open and speaking HTTP.
2. `HTTP/1.1 200 OK` confirms the request succeeded.

3. `Server: Microsoft-IIS/10.0` names the exact web server software and version, the main footprinting prize.
4. `X-Powered-By: ASP.NET` names the backend framework.
5. `Date:` just shows the server's current time; useful for little else here.

**Common mistakes:** Forgetting to press Enter twice (the request never completes, since HTTP needs a blank line to mark the end of headers); typing `GET / HTTP/1.0` with a typo, which many servers will simply reject.

**What the exam asks about it:** Flag recall is light here (`-v` is the main one shown); the exam more often tests reading the resulting `Server:` header correctly to identify the web server and version.

## 5.2 Telnet manual banner grab

**Purpose:** Identical goal to the Netcat method: connect to port 80 and manually send an HTTP request to read the `Server:` header.

**Full syntax:**

```
telnet <target> 80
```

Then type `GET / HTTP/1.0` and press Enter twice.

**Breakdown of each part:**

- `telnet` : the Telnet client program.
- `<target>` : hostname or IP address.
- `80` : destination port.

**Flag table:**

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| (none shown in notes) | Telnet here is used with no flags | Default manual banner grab | `telnet target 80` |

**Demo command in a realistic scenario:**

```
telnet www.moviescope.com 80
```

**Sample output (realistic):**

```
Trying 10.10.1.19...
Connected to www.moviescope.com.
Escape character is '^]'.
GET / HTTP/1.0

HTTP/1.1 200 OK
Content-Type: text/html
Server: Microsoft-IIS/10.0
X-Powered-By: ASP.NET
Date: Thu, 14 Mar 2024 05:57:02 GMT
```

**How to read it:** Same as the Netcat example: `Server:` and `X-Powered-By:` are the key lines to extract.

**Common mistakes:** Confusing Telnet-the-protocol (an insecure remote login protocol, a security risk if left enabled on a server) with Telnet-the-client (a generic TCP client here used safely just to send one manual HTTP request); the exam can test both meanings separately.

**What the exam asks about it:** "Which two tools can both be used for manual HTTP banner grabbing on port 80?" -> Netcat and Telnet, both correct. Also expect a trick question distinguishing "Telnet as an insecure remote access protocol that should be disabled on servers" from "Telnet as a banner-grabbing client tool," since both appear in CEH material.

## 5.3 Nmap http-userdir-enum (initial scan)

**Purpose:** Enumerate valid system usernames on an Apache server with `mod_userdir` enabled, by testing the default wordlist of usernames against the `/~username/` URL pattern.

**Full syntax:**

```
nmap -p80 --script http-userdir-enum <target>
```

**Breakdown of each part:**

- `nmap` : the Nmap scanner.
- `-p80` : only scan port 80.
- `--script http-userdir-enum` : run the specific NSE script that performs user directory enumeration.
- `<target>` : the IP address or hostname to scan.

**Flag table:**

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| `-p80` | Restrict the scan to port 80 only | When you already know HTTP is on 80 | `-p80` |

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| `--script <name>` | Load a specific NSE script | Whenever you need a specific NSE task | `--script http-userdir-enum` |
| `--script-args <k>=<v>` | Pass arguments into the loaded script | To customize wordlist or headers | see 5.4 and 5.5 |

**Demo command in a realistic scenario:**

```
nmap -p80 --script http-userdir-enum 10.10.1.22
```

**Sample output (realistic):**

```
PORT   STATE SERVICE
80/tcp open  http
| http-userdir-enum:
|   Potential Users:
|     root (403)
|     admin (200)
|_    test (404)
```

**How to read it:** Each line under "Potential Users" shows a tested username and the HTTP status code the server returned for `/~username/`. A `200` or `403` usually means the directory exists (a real user); a `404` means it does not.

**Common mistakes:** Running the script without `-p80` and letting Nmap scan all default ports first, wasting time; assuming every non-404 result is automatically a confirmed real account without manually double-checking a sample.

**What the exam asks about it:** Exact script name recall (`http-userdir-enum`) and reading the output's status codes correctly.

## 5.4 Nmap http-userdir-enum (custom wordlist)

**Purpose:** Same enumeration as 5.3, but using your own username wordlist file instead of Nmap's built-in default list.

**Full syntax:**

```
nmap -p80 --script http-userdir-enum \
  --script-args userdir.users=<wordlist.txt> <target>
```

**Breakdown of each part:**

- `--script-args userdir.users=<wordlist.txt>` : tells the `http-userdir-enum` script to read usernames from the named text file instead of its default internal list.

**Flag table:**

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| `userdir.users=<file>` | Path to a custom username wordlist | When default list misses your target's naming style | `userdir.users=staff.txt` |

**Demo command in a realistic scenario:**

```
nmap -p80 --script http-userdir-enum \
  --script-args userdir.users=staff_names.txt \
  10.10.1.22
```

**Sample output (realistic):**

```
PORT   STATE SERVICE
80/tcp open  http
| http-userdir-enum:
|   Potential Users:
|     jsmith (200)
|_    rjones (404)
```

**How to read it:** Only the usernames from `staff_names.txt` were tested this time; `jsmith` returned `200` (likely a real account), `rjones` returned `404` (not found).

**Common mistakes:** Pointing `userdir.users` to a file that does not exist or has the wrong path, which silently falls back to no results or an error depending on the Nmap version; mixing up the argument name (`userdir.users`, not `users.list` or similar).

**What the exam asks about it:** Exact argument name recall (`userdir.users`), and recognizing this is the "customized scan" variant versus the default scan in 5.3.

## 5.5 Nmap http-brute with custom User-Agent

**Purpose:** Run a brute-force style HTTP script while disguising the HTTP `User-Agent` header, to reduce the chance of simple signature-based detection blocking the scan.

**Full syntax:**

```
nmap -p80 --script http-brute \
  --script-args http.useragent="<User_Agent>" <target>
```

**Breakdown of each part:**

- `--script http-brute` : loads Nmap's generic HTTP brute-force NSE script.
- `--script-args http.useragent="<User_Agent>"` : sets a custom value for the HTTP `User-Agent` header sent with every request the script makes.

**Flag table:**

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| `http.useragent="<value>"` | Overrides the default Nmap User-Agent string | To avoid signature-based blocking | `http.useragent="Mozilla/5.0"` |

**Demo command in a realistic scenario:**

```
nmap -p80 --script http-brute \
  --script-args http.useragent="Mozilla/5.0 (Windows NT 10.0)" \
  10.10.1.22
```

**Sample output (realistic):**

```
PORT   STATE SERVICE
80/tcp open  http
| http-brute:
|   Accounts: No valid accounts found
|_  Statistics: Performed 120 guesses in 10 seconds
```

**How to read it:** The script reports whether any valid account was found, plus basic statistics on how many guesses it made and how fast.

**Common mistakes:** Forgetting quotation marks around a User-Agent string containing spaces, which breaks the shell command; assuming a changed User-Agent guarantees bypassing detection, when more advanced systems inspect behavior, not just this one header.

**What the exam asks about it:** Exact argument name recall (`http.useragent`), and understanding this is a basic evasion technique, not a guaranteed bypass.

## 5.6 Nginxpwner scan

**Purpose:** Scan an Nginx web server for common misconfigurations and known vulnerabilities, using a prepared list of URL paths to test.

**Full syntax:**

```
nano /tmp/pathlist
python3 nginxpwner.py <target_URL> /tmp/pathlist
```

**Breakdown of each part:**

- `nano /tmp/pathlist` : opens the `nano` text editor to create/edit a file listing candidate URL paths to test (one per line, typically).
- `python3 nginxpwner.py` : runs the Nginxpwner script with Python 3.
- `<target_URL>` : the full URL of the target Nginx server.
- `/tmp/pathlist` : the file of candidate paths, passed as the second argument.

**Flag table:**

| Argument position | Meaning | Example |
|---|---|---|
| 1st positional argument | Target URL to scan | `http://target.com/` |
| 2nd positional argument | Path to the path list file | `/tmp/pathlist` |

**Demo command in a realistic scenario:**

```
python3 nginxpwner.py http://192.168.10.50/ /tmp/pathlist
```

**Sample output (realistic):**

```
NGINXPWNER
A common vulnerability scanner for Nginx
Author: astark0de
NGINX out of date, version is: 1.19.0
and last version is: 1.23.1
Current exploits will be printed now:
Exploits: No Results
Shellcodes: No Results
```

**How to read it:** The version comparison line tells you the server is outdated (`1.19.0` versus the current `1.23.1`); the "No Results" lines mean no public exploits or shellcodes matched that version in the tool's local database at the time of the scan, not that the server is safe.

**Common mistakes:** Forgetting to create the `/tmp/pathlist` file first, since the script expects it as a required argument; treating "No Results" as proof of safety instead of as "nothing matched today."

**What the exam asks about it:** Recognizing Nginxpwner by name as Nginx-specific, and correctly reading a version-comparison style output to conclude "this server needs a patch."

## 5.7 Dirhunt directory crawl

**Purpose:** Crawl a website to find and analyze directories, without relying purely on a brute-force wordlist.

**Full syntax:**

```
dirhunt <target_URL>
```

**Breakdown of each part:**

- `dirhunt` : the Dirhunt program.
- `<target_URL>` : the full URL of the target site to crawl.

**Flag table:**

| Flag | Meaning | When to use | Example |
|---|---|---|---|
| (none shown in notes beyond the target URL) | Basic crawl mode | Default usage | `dirhunt http://target.com` |

**Demo command in a realistic scenario:**

```
dirhunt http://www.moviescope.com
```

**Sample output (realistic):**

```
Welcome to Dirhunt v1.0.0 using Python 3.11.2
[200] http://www.moviescope.com/
      (HTML document)
[404] http://www.moviescope.com/css/images/
      (Not Found)
[404] http://www.moviescope.com/js/js/
      (Not Found)
```

**How to read it:** Each line shows the HTTP status code in brackets, the URL tested, and a short note; `[200]` lines are confirmed existing resources worth reviewing further, `[404]` lines are ruled out.

**Common mistakes:** Expecting Dirhunt to behave exactly like a pure wordlist brute-forcer; it instead follows links and page assets, so results depend heavily on how much the target site links to itself.

**What the exam asks about it:** Recognizing Dirhunt as the tool that finds directories "without bruteforce," by following links rather than guessing blindly.

## 6. Ports, Protocols, Standards and Acronyms

| Item | Simple purpose | Default port/ layer | Known weakness | Related attack/ tool |
|---|---|---|---|---|
| HTTP | Carries plain web traffic | Port 80, App layer | No encryption, headers can be injected | Response splitting, cache poisoning |
| HTTPS (SSL/TLS) | Encrypts web traffic | Port 443, App layer | Weak/self-signed/ expired certs | Misconfigured or default certificates |

| Item | Simple purpose | Default port/ layer | Known weakness | Related attack/ tool |
|---|---|---|---|---|
| HTTP/2 | Newer, multiplexed HTTP version | Same ports as HTTP/HTTPS | Frame-handling bugs (CONTINUATION flood) | HTTP/2 continuation flood, CVE-2024-27316 |
| SSH | Encrypted remote login/ tunnel | Port 22, App layer | Weak/reused passwords, brute-forceable | SSH brute force attack |
| Telnet | Plain-text remote login (insecure) | Port 23, App layer | No encryption at all | verify: insecure protocol to disable |
| FTP | File transfer | Ports 20/21, App layer | Often plain text credentials | verify: default/ anonymous FTP accounts |
| SMB | Windows file/ printer sharing | Port 445 (also 139), App layer | Exposed on internet = high risk | Block on internet-facing servers |
| NetBIOS | Legacy name resolution/ sharing | Ports 137-139, App layer | Legacy, abused for enumeration | Block on internet-facing servers |
| DNS | Resolves domain names to IPs | Port 53, App layer | Server can be hijacked or cache poisoned | DNS server hijacking |
| WebDAV | Remote file management over HTTP | Uses HTTP/HTTPS ports | Can expose file write access if misused | Disable if unused |
| POP3 | Retrieves email from a mail server | Port 110, App layer | Often plain text, listed as insecure | Secure with IPsec if required |
| SMTP | Sends email between servers | Port 25, App layer | Plain text by default, listed as insecure | Secure with IPsec if required |
| IMAP | Retrieves/syncs email from a server | Port 143, App layer | Plain text variant insecure | Secure with IPsec if required |
| ICMP | Network diagnostics (ping, errors) | Network layer (no port) | Can be abused for recon/flood | Block unnecessary ICMP traffic |
| RFC 2616 | Defines the HTTP/ 1.1 standard | Standard document | Violated by CRLF-injected requests | HTTP response splitting |

| Item | Simple purpose | Default port/ layer | Known weakness | Related attack/ tool |
|---|---|---|---|---|
| ICANN | Oversees global domain registration | Governance body | Weak registrar account security | DNS hijacking countermeasure |

**ADDED CONTEXT:**HTTP/2 does not use a different port from HTTP or HTTPS; it negotiates on top of the same ports (80 or 443, typically via TLS ALPN negotiation on 443). The exam cares more about HTTP/2's framing behavior (HEADERS, CONTINUATION, END_HEADERS) than about a separate port number.

# 7. Attack vs Countermeasure and Look-alike Comparisons

## 7.1 Attack vs Countermeasure

| Attack | Primary countermeasure |
|---|---|
| Directory traversal | Canonicalize/validate file paths; reject decoded `../` |
| Web server misconfiguration | Disable directory browsing; restrict status pages |
| HTTP response splitting | Disallow CRLF (`%0d`/`%0a`) in header-bound input |
| Web cache poisoning | Patch server; avoid shared TCP connections per virtual host |
| SSH brute force | Strong/unique passwords or key auth; lockout after failures |
| HTTP/2 continuation flood | Patch HTTP/2 stack; limit CONTINUATION frame count/size |
| FrontJacking | Validate/whitelist Host headers at the reverse proxy |
| DNS server hijacking | Registrar-Lock; DNS monitoring; secure registrant account |
| Nginx alias traversal | Match trailing slashes on `location` and `alias` directives |
| mod_userdir enumeration | Disable mod_userdir, or hide valid vs invalid |
| Outdated web server/software | Scan, patch, test in staging, keep a back-out plan |
| Over-privileged accounts/services | Least privilege; remove unused accounts/ modules/shares |

### 7.2 Look-alike Comparisons

- **Directory traversal vs directory brute forcing.** Difference: traversal escapes the web root using `../`; brute forcing guesses names of files/folders inside or around the root. Tell them apart: look for `../` or an encoded form versus a wordlist or crawler testing many guessed paths.
- **DNS server hijacking vs web cache poisoning.** Difference: hijacking changes the DNS server's own records; poisoning corrupts a separate cache's stored content instead. Tell them apart: ask which system was actually manipulated, the DNS server or a cache entry.
- **HTTP response splitting vs web cache poisoning.** Difference: splitting is the CRLF injection mechanism itself; poisoning is often the end result built on top of that mechanism. Tell them apart: splitting answers "how," poisoning answers "so what happened next."
- **Nginx alias traversal vs generic directory traversal.** Difference: alias traversal is a specific Nginx config bug (a trailing slash mismatch); generic traversal is a bug in application or script code. Tell them apart: look for an `alias` directive and `location` block versus a vulnerable script parameter.
- **Vulnerability scanner vs pen-testing tool.** Difference: a scanner detects and reports; a pen-testing tool actively exploits and validates. Tell them apart: Acunetix/WebInspect report findings, CORE Impact/Cobalt Strike exploit them.
- **Netcat/Telnet banner grab vs Shodan.** Difference: Netcat and Telnet make a live, direct connection to the target right now. Tell them apart: Shodan instead queries a search engine's pre-collected banner database, with no direct contact at query time.
- **Telnet (protocol) vs Telnet (client tool).** Difference: the protocol is an insecure remote login method that should be disabled on servers. Tell them apart: the client tool is just a generic TCP client, safely used here only for manual HTTP banner grabs.
- **mod_userdir enumeration vs directory brute forcing.** Difference: mod_userdir enumeration guesses usernames through `/~user/`; brute forcing guesses general folder or file names. Tell them apart: look specifically for the `/~username/` URL pattern.

## 8. Exam Cheat Sheet

**Keyword -> Answer pairs:**

- "endless CONTINUATION frames", "HEADERS without END_HEADERS" -> HTTP/2 continuation flood (CVE-2024-27316 class).
- "CRLF", "%0d%0a", "two responses from one request" -> HTTP response splitting.
- "poisoned cache entry", "intermediate web cache" -> web cache poisoning.

- "Nginx reverse proxy", "shared hosting", "Host header" + CRLF + XSS -> FrontJacking.
- "alias directive", "missing trailing slash" -> Nginx alias traversal (tool: Kyubi).
- "dot-dot-slash", "../", "outside web root" -> directory traversal.
- "compromises the DNS server and changes settings" -> DNS server hijacking (not cache poisoning, not phishing).
- "search engine for internet-connected devices", "banners" -> Shodan.
- "/~username/", "mod_userdir" -> Nmap `http-userdir-enum` script.
- "crawler, finds directories without a wordlist" -> Dirhunt.
- "common misconfiguration scanner for Nginx specifically" -> Nginxpwner.
- "searchable archive of public exploit code, EDB-ID, CVE" -> Exploit-DB.
- "kernel mode HTTP.SYS, user mode W3WP.exe" -> IIS architecture.
- "master process, worker processes, non-blocking I/O" -> Nginx architecture.
- "modular, mod_ssl, mod_auth, mod_rewrite, mod_proxy" -> Apache architecture.
- "Server:" HTTP header -> banner grabbing result (names the exact software and version).
- "Registrar-Lock", "ICANN accredited registrar" -> DNS hijacking countermeasure.
- "do not install IIS on a domain controller" -> general hardening fact, very frequently tested.
- "never trail by more than two service packs" -> patch management fact, memorize the number two.

**Must-memorize lists:**

- The six-step Web Server Attack Methodology, in order: Information Gathering -> Web Server Footprinting -> Website Mirroring -> Vulnerability Scanning -> Session Hijacking -> Web Server Passwords Hacking.
- The IIS Web Server Core pipeline order: Begin Request Processing -> Authentication -> Authorization -> Cache Resolution -> Handler Mapping -> Handler Pre-execution -> Release State -> Update Cache -> Update Log -> End Request Processing.
- Seven reasons web servers are compromised (section 3.2): improper permissions, unnecessary default/backup/sample files, misconfig, software bugs, exposed admin/debug functions, weak/self-signed certificates, no dedicated server.
- Six impacts of web server attacks (section 3.2): compromised accounts, defacement, secondary attacks, root access elsewhere, data theft/tampering, reputational damage.

**Numbers and defaults to remember:**

- Port 80 = HTTP, Port 443 = HTTPS/SSL/TLS: the only two ports that should typically remain open on a hardened public web server.
- "No more than two service packs behind" is the specific patch-lag limit quoted in the notes.

- `http-userdir-enum` and `http.useragent` and `userdir.users` are the exact Nmap NSE script and argument names to recall precisely.

**"Pick the BEST option" rules for this module:**

- If a scenario could be both "misconfiguration" and a specific named attack (response splitting, cache poisoning, FrontJacking), the more specific named attack is usually the best answer if its exact mechanism (CRLF injection, Host header trust, alias trailing slash) is clearly described; otherwise default to the general "misconfiguration" category.
- If asked "what should be done first" among patch, test, deploy, back-out plan -> the order is always: review documentation, test in non-production, deploy with monitoring, keep the back-out plan ready the entire time (it is prepared before deployment, not created after something breaks).
- If asked to choose between a vulnerability scanner and a pen-testing tool for "confirming a finding is truly exploitable," the pen-testing tool (CORE Impact, Cobalt Strike, mitmproxy, Fuxploider) is the better answer, since scanners detect but do not always confirm real-world exploitability.

## 9. Practice Questions

Each question is followed immediately by its answer and explanation.

**Q1.** A small company runs its public website on IIS with the original sample pages and an unused FTP service still installed, and the box also stores the company's shared files. An attacker finds and exploits the sample pages. According to this module, what is the single underlying category of weakness that let this happen?

A. A zero-day bug in IIS itself
B. Web server misconfiguration (unnecessary default/sample files, no dedicated server)
C. A DNS server hijacking attack
D. A web application SQL injection flaw

**Answer: B.** The scenario matches two of the seven listed causes of web server compromise directly: unnecessary default/sample files left installed, and not using a dedicated server for web services. A is wrong because no zero-day bug is described, only leftover defaults. C is wrong because no DNS records were touched. D is wrong because no SQL or application-layer flaw is mentioned, only leftover sample files and a shared server role.

**Q2.** Which Apache module is responsible for forwarding a request to a separate application server to retrieve dynamic content?

A. mod_ssl
B. mod_auth

C. mod_rewrite
D. mod_proxy

**Answer: D.** `mod_proxy` is the module that lets Apache forward requests to another server, exactly as shown in the architecture diagram. A (`mod_ssl`) handles encryption, not forwarding. B (`mod_auth`) handles authentication checks. C (`mod_rewrite`) rewrites URLs; it does not forward a request to a different server.

**Q3.** Which IIS component runs in kernel mode and is responsible for first receiving every incoming HTTP request on the server?

A. W3WP.exe
B. Windows Activation Service (WAS)
C. HTTP.SYS
D. svchost.exe

**Answer: C.** `HTTP.SYS` is explicitly shown as the kernel-mode HTTP Protocol Stack that receives requests first. A (`W3WP.exe`) is the user-mode worker process that does the actual page handling. B (WAS) runs in user mode and manages activation of application pools. D (`svchost.exe`) hosts the WWW Service in user mode, not kernel mode.

**Q4.** A vulnerability report describes an attacker opening a TCP connection to an Apache server, sending a HEADERS frame without the END_HEADERS flag, and then flooding the connection with CONTINUATION frames until the server runs out of memory and crashes. Which attack is this?

A. HTTP response splitting
B. Web cache poisoning
C. HTTP/2 continuation flood
D. SSH brute force

**Answer: C.** This exactly matches the HTTP/2 continuation flood attack steps from the module (and the real CVE-2024-27316 class of issue): HEADERS without END_HEADERS, followed by endless CONTINUATION frames, leading to memory exhaustion. A is wrong because response splitting uses CRLF-injected input to generate two HTTP responses, not HTTP/2 frame flooding. B is wrong because no cache is mentioned. D is wrong because this is not a login credential guessing attack.

**Q5.** A security architect describes a web server that uses one supervising process to manage several worker processes, each handling many connections at once using non-blocking I/O inside a single thread. Which web server architecture is being described?

A. Apache
B. IIS

C. Nginx
D. Tomcat

**Answer: C.** "Master process... worker processes... non-blocking I/O... single-threaded loop" is the exact Nginx architecture described in the notes. A (Apache) is module-based, not master/worker-based. B (IIS) splits work between kernel mode and user mode, not master/worker. D (Tomcat) is not covered in this module's architecture slides.

**Q6.** An organization discovers that visitors to its real domain are being silently sent to a fake, attacker-controlled website. Investigation shows the attacker gained administrative access to the organization's authoritative DNS server and directly edited its DNS records. Which attack is this, and which is the closest but incorrect look-alike an exam might also offer?

A. Web cache poisoning; DNS server hijacking is the look-alike
B. DNS server hijacking; web cache poisoning is the look-alike
C. FrontJacking; directory traversal is the look-alike
D. Phishing; SQL injection is the look-alike

**Answer: B.** The attacker directly compromised and edited the authoritative DNS server's own settings, which is DNS server hijacking by definition. Web cache poisoning is the closest look-alike because it also ends with a victim seeing fake content, but it works by corrupting a cache's stored entry, not by changing DNS records. A reverses the correct attack and look-alike. C and D name attacks that do not match a DNS-record-editing scenario at all.

**Q7.** A developer blocks the literal text `../` in a file-path parameter to prevent directory traversal, but a penetration tester still manages to read files outside the web root. What is the most likely reason the filter failed?

A. The web server does not actually support path-based requests
B. The tester used a completely different attack, such as SQL injection
C. The tester used an encoded form of the traversal sequence, such as `%5c` or `%2e%2e%2f`, which the literal-text filter did not catch
D. Directory traversal cannot be prevented by any filter

A, B, and D are each wrong for a stated reason below.

**Answer: C.** The module's own example URL uses `%5c` (encoded backslash) specifically to get past a filter that likely only checked for the plain `../` or `..\` text; this is the standard encoding-bypass trap. A is wrong because the server clearly does support path-based requests, since the traversal worked. B is wrong because the scenario is clearly a file-path/traversal issue, not a database query issue. D is wrong because proper server-side canonicalization and validation, applied after decoding, can prevent this; it is not unpreventable.

**Q8.** An auditor finds the following in a site's `web.config` file:

```
<system.webServer>
  <directoryBrowse enabled="true" />
</system.webServer>
```

What risk does this setting introduce?

A. It disables HTTPS encryption for the site
B. It allows visitors to see a full file listing of a folder when no default page exists, which can expose files not meant to be public
C. It enables SQL injection on every form on the site
D. It automatically redirects all HTTP traffic to HTTPS

**Answer: B.** `directoryBrowse enabled="true"` is exactly the IIS directory listing misconfiguration shown in the notes, which exposes folder contents including files nobody intended to publish. A, C, and D describe unrelated settings (TLS configuration and SQL injection are not affected by this flag at all).

**Q9.** A web application takes a `author` URL parameter and places it directly into a `Set-Cookie` header with no filtering. An attacker sends a link where the parameter value contains carriage-return and line-feed characters followed by extra header-like text. What class of attack does this enable?

A. HTTP response splitting
B. SSH brute force
C. Directory traversal
D. DNS server hijacking

**Answer: A.** Injecting CRLF characters into a value that lands in an HTTP header is the defining mechanic of HTTP response splitting, which can cause the server to emit a second, attacker-controlled response. B, C, and D involve completely different mechanisms (password guessing, path traversal, DNS record tampering) and none of them match CRLF header injection.

**Q10.** During an incident, investigators find that an attacker first sent a request to flush a page from a shared web cache, then sent a specially crafted request that generated two HTTP responses, then requested the real page again so the cache stored the second, malicious response under the real page's address. What is this attack called?

A. HTTP/2 continuation flood
B. Web cache poisoning
C. FrontJacking
D. mod_userdir enumeration

**Answer: B.** This step-by-step sequence (flush cache entry, generate two responses via CRLF injection, request again to force a fresh cache entry pointing at the malicious

response) is exactly the web cache poisoning attack from the notes. A is a completely different HTTP/2 framing-based DoS. C involves a reverse proxy Host header, not a cache. D is about enumerating usernames, unrelated to caching.

**Q11.** Which of the following is the BEST defense against SSH brute force attacks on a public-facing server, according to this module's guidance?

A. Disabling HTTPS on the web server
B. Enforcing strong, unique passwords (or key-based auth) plus login attempt limits and monitoring
C. Switching the web server from Apache to Nginx
D. Enabling directory browsing to monitor login attempts

**Answer: B.** The module's countermeasures emphasize strong password policy, account lockout/limits on failed attempts, and auditing/alerts for login failures. A is unrelated to SSH. C changes the web server software, which does not affect SSH credential strength at all. D (directory browsing) is an unrelated misconfiguration, not a login protection.

**Q12.** A server shows rapidly climbing memory usage immediately after receiving a HEADERS frame without an END_HEADERS flag, followed by a stream of CONTINUATION frames from the same client. What is the best immediate next step?

A. Ignore it, since HTTP/2 is always encrypted and therefore safe
B. Apply/verify the HTTP/2 patch that limits CONTINUATION frame count and size per stream, and rate-limit or drop the offending connection
C. Switch the server from HTTP/2 back to HTTP/1.0 permanently as the only fix
D. Increase the server's total available memory as a permanent fix

**Answer: B.** The documented fix for this exact pattern (matching CVE-2024-27316-class issues) is patching the HTTP/2 implementation to cap CONTINUATION frames and applying connection-level rate limiting. A is wrong because encryption has nothing to do with this frame-handling bug. C is an extreme, unnecessary step when a targeted patch exists. D only delays the crash under a larger flood; it does not fix the root cause.

**Q13.** A shared-hosting provider's Nginx reverse proxy trusts the client-supplied Host header to decide which backend to route a request to. An attacker injects a crafted Host header combined with CRLF and XSS payloads to redirect another customer's traffic to an attacker-controlled server. What is this attack called?

A. Directory traversal
B. FrontJacking
C. SSH brute force
D. Web server password cracking

**Answer: B.** This matches FrontJacking exactly: a poorly configured Nginx reverse proxy in shared hosting, combining CRLF injection, HTTP header injection, and XSS to hijack routing via the Host header. A, C, and D describe unrelated mechanisms (file path escape, password guessing, credential cracking).

**Q14.** Put the six steps of the Web Server Attack Methodology in the correct order.

A. Vulnerability Scanning, Information Gathering, Footprinting, Mirroring, Password Hacking, Session Hijacking
B. Information Gathering, Web Server Footprinting, Website Mirroring, Vulnerability Scanning, Session Hijacking, Web Server Passwords Hacking
C. Website Mirroring, Information Gathering, Vulnerability Scanning, Footprinting, Password Hacking, Session Hijacking
D. Session Hijacking, Password Hacking, Footprinting, Mirroring, Information Gathering, Vulnerability Scanning

**Answer: B.** This is the exact order given in the module: Information Gathering, Web Server Footprinting, Website Mirroring, Vulnerability Scanning, Session Hijacking, Web Server Passwords Hacking. A, C, and D all scramble this specific sequence.

**Q15.** An organization wants to stop attackers from learning its internal directory structure through its own `robots.txt` file. What is the BEST fix?

A. Delete robots.txt entirely and rely on that alone
B. Add real authentication and access control to sensitive directories, instead of relying on robots.txt to hide them
C. List the sensitive directories in robots.txt using a different file name so crawlers cannot find it
D. Nothing can be done; robots.txt is mandatory and cannot be changed

**Answer: B.** robots.txt is only an advisory instruction to well-behaved crawlers, not an access control; the correct fix is to actually protect sensitive directories with authentication and access control. A is incomplete and also does not undo exposure to anyone who already saw it. C just moves the same voluntary-only protection elsewhere; it does not add real access control. D is false; the file is entirely optional and fully editable by the site owner.

**Q16.** Which flag or step is required to correctly complete a manual HTTP banner grab using Netcat or Telnet on port 80?

A. Press Enter once after typing `GET / HTTP/1.0`
B. Press Enter twice after typing `GET / HTTP/1.0`, so the blank line signals the end of the HTTP request
C. Add the flag `-ssl` to force an encrypted connection
D. Type `POST / HTTP/1.0` instead of `GET`

**Answer: B.** Both the Netcat and Telnet examples in the module show pressing Enter twice; the blank line is what tells the server the headers are finished. A (pressing Enter only once) leaves the request incomplete. C is not a real Netcat/Telnet flag for this purpose, and port 80 is plain HTTP, not HTTPS. D is unnecessary; a simple `GET` is what is used in the module's examples.

**Q17.** Which exact Nmap script-args name supplies a custom username wordlist file to the `http-userdir-enum` script?

A. `users.list`
B. `userdir.users`
C. `enum.wordlist`
D. `http.users`

**Answer: B.** The module gives the exact syntax `--script-args userdir.users=<wordlist.txt>`. A, C, and D are plausible sounding but incorrect argument names not used by this script.

**Q18.** Which Shodan filter would best help an attacker find IIS servers located specifically in the United States?

A. `http.title:"IIS Windows Server" country:"US"`
B. `ssl:"Company Inc." http.title:"IIS"`
C. `http.title:"IIS7" port:80`
D. `ssl.cert.subject.CN:"company.in" http.title:"IIS"`

**Answer: A.** This filter combines the IIS title match with a country filter for the US, exactly as shown in the notes. B filters by SSL certificate issuer name, not country. C filters by IIS7 specifically on port 80, with no country filter. D filters by a certificate's common name, not country.

**Q19.** A pentester wants a tool that finds and analyzes a website's directories by crawling its links and page assets, rather than purely guessing names from a wordlist. Which tool fits best?

A. Nginxpwner
B. Dirhunt
C. Exploit-DB
D. Kyubi

**Answer: B.** Dirhunt is described specifically as a crawler optimized to search and analyze directories, notable for finding content by following links rather than relying only on brute-force wordlists. A (Nginxpwner) scans for Nginx misconfigurations, not general directory discovery. C (Exploit-DB) is an exploit code archive, not a crawler. D (Kyubi) targets a specific Nginx alias traversal misconfiguration, not general directory crawling.

**Q20.** A scan with Nginxpwner reports: "NGINX out of date, version is: 1.19.0 and last version is: 1.23.1" followed by "Exploits: No Results". What is the correct interpretation?

A. The server is fully safe since no exploit was found
B. The server is outdated and should be patched, even though no public exploit was found in the tool's database at scan time
C. The scan failed and should be re-run
D. Version 1.19.0 is actually newer than 1.23.1

**Answer: B.** The version comparison clearly shows the server is behind the latest release; "No Results" for exploits only means nothing matched in that specific lookup at that time, not that the software is safe or unexploitable by some other means. A wrongly treats absence of a known exploit as proof of safety. C is incorrect; the scan completed and reported results normally. D misreads the version numbers; 1.23.1 is the newer release.

**Q21.** A pentester has confirmed through banner grabbing that a target runs Apache with the `mod_proxy_cluster` module enabled, an older version. What is the most appropriate next research step to find a usable, already-written exploit for this exact configuration?

A. Search Shodan for the target's IP address
B. Search Exploit-DB for "Apache mod_proxy_cluster" and cross-check the CVE and version against the target
C. Run Nginxpwner against the target
D. Run the `http-userdir-enum` Nmap script

**Answer: B.** Exploit-DB is the searchable archive of public, working exploit code described in the notes, and searching by software and module name, then verifying the CVE and version, is exactly the documented workflow. A (Shodan) is for finding exposed devices, not exploit code. C (Nginxpwner) is Nginx-specific, not relevant to an Apache target. D is a username enumeration script, unrelated to finding exploit code for a known module.

**Q22.** An Nginx configuration contains:

```
location /i {
  alias /data/w/images/;
}
```

A tester finds that adding extra `../` and `./` segments after `/i` in the URL eventually returns files outside `/data/w/images/`. What is the root cause, and which tool is purpose-built to find this class of bug?

A. A SQL injection flaw in the backend; tool is Acunetix
B. A missing trailing slash mismatch between the `location` path and the `alias` path;

tool is Kyubi
C. An expired SSL certificate; tool is Shodan
D. A weak SSH password; tool is Hydra

**Answer: B.** This is the exact Nginx alias traversal pattern from the notes: `location /i` lacks a trailing slash while the `alias` value has one, which Kyubi is specifically built to discover and exploit. A, C, and D describe unrelated vulnerability classes (SQL injection, certificate expiry, SSH credentials) that do not match a trailing-slash alias misconfiguration at all.

**Q23.** According to this module's patch management guidance, what is the maximum number of service packs an organization should ever fall behind on a server?

A. Zero; every single patch must be applied the same day it releases
B. Two
C. Five
D. There is no stated limit

**Answer: B.** The module states explicitly: "never trail by more than two service packs." A is too extreme and conflicts with the module's own advice to first test patches in non-production before deployment. C overstates the allowed gap. D is incorrect; a specific number (two) is given.

**Q24.** Why does the module specifically recommend never installing IIS on a domain controller?

A. IIS cannot technically run on a domain controller at all
B. A web server compromise on that same machine could cascade into a compromise of the entire domain's authentication system
C. Domain controllers do not support HTTP traffic
D. It would make the website load more slowly

**Answer: B.** The risk is cascading impact: if the web server role is compromised, an attacker on the same machine as a domain controller could reach the domain's core authentication data. A is factually wrong; it is technically possible, just not recommended. C is false; domain controllers can serve HTTP traffic. D is not the stated reason in the module; the concern is security isolation, not performance.

**Q25.** An attacker wants to speed up passive reconnaissance against a company's public-facing IIS infrastructure using an AI chatbot before ever touching the target. Which of the following is the most appropriate and realistic AI-assisted step, consistent with this module's AI angle guidance?

A. Asking the AI to directly log into the target's admin panel
B. Asking the AI to generate a precise Shodan search filter syntax for the attacker's plain-language goal (for example, finding IIS servers tied to a given company's SSL certificate)

C. Asking the AI to automatically launch a live brute-force attack against the target
D. Asking the AI to physically break into the company's data center

**Answer: B.** The module's AI angle for Shodan recon is exactly this: using an AI assistant to turn a plain-language research goal into correct Shodan filter syntax, which remains a passive, no-contact research step. A, C, and D all describe an AI directly performing active intrusion or physical actions, which is not how the AI angle in this module is framed (helping craft queries and review/automate research, not autonomously attacking).

**Q26.** A defensive security team wants to use AI to help catch HTTP response-splitting bugs before release. Which prompt best matches the module's AI angle guidance for this specific attack?

A. "Write me a working HTTP response splitting exploit against our production login page right now."
B. "Review this code for CRLF injection risk in any HTTP header or cookie value built from user input."
C. "Generate a phishing email pretending to be our own IT department."
D. "Delete all logging code so the website runs faster."

**Answer: B.** This matches the module's suggested defensive AI prompt almost exactly: reviewing code for CRLF injection risk in header or cookie-building logic. A asks the AI to actively attack a live production system, which is unsafe and not the defensive use case described. C is unrelated social engineering content generation. D would remove security visibility (logging) rather than help find a response-splitting bug.

**Q27.** A security team needs a tool that will both detect a potential vulnerability on a web server and then actively confirm it is truly exploitable in a controlled test. Which category of tool is BEST suited for the "confirm it is truly exploitable" half of that task, as distinguished in this module?

A. A vulnerability scanner such as Acunetix or WebInspect
B. A penetration-testing tool such as CORE Impact or Cobalt Strike
C. A DNS monitoring service
D. A patch management checklist

**Answer: B.** The module distinguishes vulnerability scanners (detect and report, like Acunetix and WebInspect) from pen-testing tools (actively exploit and validate, like CORE Impact and Cobalt Strike); confirming real-world exploitability is squarely the pen-testing tool's job. A is the detection half, not the confirmation half. C and D are unrelated to exploit validation entirely.

**Q28.** Which exact Nmap flag restricts a scan to only port 80 before running the `http-userdir-enum` script?

A. `--port 80`
B. `-p80`
C. `-port=80`
D. `--only-port 80`

**Answer: B.** The module's exact syntax is `nmap -p80 --script http-userdir-enum <target>`. A, C, and D are not valid Nmap flag syntax for restricting scanned ports.

**Q29.** Following this module's port-hardening guidance, which two ports should typically remain open on a properly hardened public web server?

A. 20 and 21
B. 80 and 443
C. 23 and 110
D. 25 and 53

**Answer: B.** The module specifically states to limit inbound traffic to port 80 for HTTP and port 443 for HTTPS (SSL). A (FTP), C (Telnet and POP3), and D (SMTP and DNS) are all services the hardening guidance elsewhere recommends restricting, securing, or blocking as unnecessary on a plain public web server.

**Q30.** What is the key difference between "Telnet" as referenced in the protocol-hardening countermeasures slide and "Telnet" as used for manual banner grabbing in the footprinting section of this module?

A. There is no difference; both refer to exactly the same risk
B. As a protocol, Telnet is an insecure remote login method that countermeasures recommend disabling or securing on a server; as a client tool, it is simply used here to send one manual HTTP request for research purposes
C. Telnet the protocol only works on Windows; Telnet the client tool only works on Linux
D. Banner grabbing with Telnet is illegal, but using Telnet as a remote login protocol is always legal

**Answer: B.** This captures the exact distinction: Telnet-the-protocol (an insecure way to remotely log into a system, which should be disabled or secured per the hardening guidance) is a different concept from Telnet-the-client-tool (just a generic TCP client used here to send a manual `GET` request for footprinting research). A incorrectly collapses two distinct concepts into one. C invents a false platform restriction. D incorrectly frames legality, which depends entirely on authorization, not on which use case is being discussed.

# 10. Flashcards

Q: Why are web servers more exposed than internal servers? | A: They must stay reachable from the public internet to serve anyone. Q: Name the seven reasons web servers are compromised. | A: Bad permissions, leftover default/sample files, misconfig, software bugs, exposed admin/debug functions, weak/default certificates, no dedicated server. Q: Which Apache module handles encryption? | A: mod_ssl. Q: Which Apache module forwards requests to another server? | A: mod_proxy. Q: Which Apache module rewrites URLs? | A: mod_rewrite. Q: What runs in kernel mode in IIS? | A: HTTP.SYS, the HTTP Protocol Stack. Q: What runs in user mode and actually processes an IIS request? | A: W3WP.exe, the worker process. Q: What manages IIS application pool activation? | A: Windows Activation Service (WAS). Q: What is Nginx's core architecture pattern? | A: One master process supervising many worker processes using non-blocking I/O. Q: What CVE class matches the HTTP/2 continuation flood attack? | A: CVE-2024-27316 class, memory exhaustion from endless CONTINUATION frames. Q: What sequence does the HTTP/2 continuation flood attack rely on? | A: HEADERS frame without END_HEADERS, then endless CONTINUATION frames. Q: What does FrontJacking combine? | A: CRLF injection, HTTP request header injection, and XSS against a misconfigured Nginx reverse proxy. Q: What does FrontJacking manipulate to hijack routing? | A: The HTTP Host header. Q: What characters does directory traversal rely on? | A: ../ (dot-dot-slash), or encoded forms like %5c and %2e%2e%2f. Q: What does IIS's directoryBrowse="true" setting expose? | A: A full file listing of a folder when no default page exists. Q: What characters define HTTP response splitting? | A: CRLF, carriage return and line feed (%0d and %0a). Q: What is the end result of web cache poisoning? | A: A shared cache stores an attacker's page under a legitimate page's address. Q: Why is SSH brute force still possible even though SSH is encrypted? | A: Encryption protects data in transit, not the strength of the login password being guessed. Q: What are the six steps of the Web Server Attack Methodology? | A: Information Gathering, Footprinting, Website Mirroring, Vulnerability Scanning, Session Hijacking, Password Hacking. Q: Is robots.txt a security control? | A: No, it is only a voluntary instruction for well-behaved crawlers. Q: What HTTP header reveals the exact web server software in a banner grab? | A: The Server: header. Q: What two tools does the module show for manual HTTP banner grabbing? | A: Netcat and Telnet. Q: What Nmap script enumerates usernames via mod_userdir? | A: http-userdir-enum. Q: What script-args name supplies a custom username wordlist to that script? | A: userdir.users. Q: What script-args name sets a custom User-Agent in Nmap's http-brute? | A: http.useragent. Q: What does Shodan search by, instead of page content like Google? | A: Device banners. Q: What tool crawls a site to find directories by following links, not brute force? | A: Dirhunt. Q: What tool is purpose-built to scan for Nginx misconfigurations? | A: Nginxpwner. Q: What companion tool analyzes an exposed nginx.conf file for misconfigurations? | A: Gixy. Q: What is Exploit-DB? | A: A public searchable archive of proof-of-concept and working exploit code. Q: What Nginx config mistake does Kyubi find and exploit? | A: A trailing slash mismatch in the alias directive versus the location path. Q: What is the maximum recommended

service pack lag per this module? | A: Never more than two service packs behind. Q: Why should IIS never be installed on a domain controller? | A: A web server compromise could cascade into a full domain authentication compromise. Q: Which two ports should remain open on a hardened public web server? | A: 80 (HTTP) and 443 (HTTPS). Q: What is the difference between a vulnerability scanner and a pen-testing tool? | A: A scanner detects and reports; a pen-testing tool actively exploits and validates. Q: Name four web server pen-testing tools from this module. | A: CORE Impact, Cobalt Strike, Fuxploider, mitmproxy. Q: Name three web server security/defense tools from this module. | A: NetIQ Secure Configuration Manager, Sophos Intercept X for Server, UpGuard. Q: What does DAST stand for and what does it test? | A: Dynamic Application Security Testing; it tests a running application from the outside. Q: What should be disallowed in web application developer input handling to stop response splitting? | A: Carriage return and line feed characters (%0d, %0a) in header-bound input. Q: What is the proxy-server-side countermeasure against response splitting and cache poisoning? | A: Avoid sharing TCP connections among different clients; use separate connections per virtual host. Q: What should an organization do to its domain registrar account to help prevent DNS hijacking? | A: Use an ICANN-accredited registrar, enable Registrar-Lock, and safeguard the registrant account.

## 11. Glossary

ACL (Access Control List): a list of permissions attached to a resource defining who can access it and how. Alias directive: an Nginx setting that maps a URL location to a different filesystem path. Application pool: an IIS isolation boundary so one site's crash does not affect another. Banner grabbing: reading a service's self-reported identity, such as the Server header, to learn its software and version. Brute force attack: trying many or all possible password guesses until the correct one is found. Byte code: compiled code that can sometimes be decompiled to reveal embedded secrets if not handled carefully. CAS (Code Access Security): a .NET security model restricting what code can do based on trust level. CONTINUATION frame: an HTTP/2 frame carrying extra header data after a HEADERS frame. CRLF: Carriage Return, Line Feed; the characters marking the end of a line in HTTP, abused in header injection. Dedicated server: a machine used for exactly one role, limiting the blast radius of a compromise. Directory browsing/listing: a server-generated page showing every file in a folder. Directory traversal: using ../ sequences to escape the intended web root directory. DNS mapping: the record linking a domain name to an IP address. DNS server hijacking: compromising a DNS server and changing its settings to redirect users to a fake site. Dot-dot-slash: the ../ sequence used to move up one directory level. EDB-ID: Exploit-DB's own internal identifier for an exploit entry. END_HEADERS flag: the HTTP/2 flag signaling that no more header data is coming. Exploit-DB: a public searchable archive of exploit code, organized by software and CVE. FrontJacking: an attack combining CRLF injection, header injection, and XSS against a misconfigured Nginx reverse proxy. Gixy: a tool that analyzes nginx.conf files for misconfigurations. Host header: the HTTP

header telling a server which website a client wants to reach. HTTP.SYS: the kernel-mode HTTP protocol stack in Windows/IIS. HTTP response splitting: injecting CRLF characters so a server emits two responses instead of one. ICANN: the organization overseeing global domain name registration policy. inetpub: the default root folder for IIS websites on Windows. IPsec: a protocol suite that authenticates and encrypts IP packets. ISAPI filter: a module that processes every IIS request before the final handler. Kyubi: a tool that discovers and exploits Nginx alias traversal misconfigurations. Least privilege: giving an account or process only the access it actually needs. Load balancer: a server that spreads requests across multiple backend servers. Metabase: older IIS configuration storage, later replaced in newer IIS versions (verify exact version). mod_proxy: the Apache module that forwards requests to another server. mod_rewrite: the Apache module that rewrites incoming URLs. mod_ssl: the Apache module that handles TLS/SSL encryption. mod_userdir: the Apache module exposing per-user directories at /~username/. Native module (IIS): a module built into IIS itself, such as static file handling. NetBIOS: a legacy Windows name-resolution and sharing protocol. Nginxpwner: a Python tool for scanning Nginx misconfigurations and vulnerabilities. Non-blocking I/O: handling many network connections without waiting idle on any single one. Reverse proxy: a server sitting in front of backends, forwarding and hiding them from clients. RFC 2616: the specification document defining HTTP/1.1. Robots.txt: a public text file advising crawlers which paths not to index. SAM (Security Accounts Manager): the Windows database storing local account/password data. Script mapping: the IIS setting connecting a file extension to its execution engine. Shodan: a search engine indexing internet-connected devices by their banners. SSH tunnel: an encrypted connection between two hosts used to transport data securely. Trailing slash mismatch: an Nginx config error where location and alias paths do not consistently end in /. Virtual host: one physical server hosting multiple distinct websites or domains. WAS (Windows Activation Service): the IIS component that manages application pool activation. W3WP.exe: the IIS user-mode worker process where site code actually runs. WebDAV: an HTTP extension allowing remote file management on a web server. Web cache poisoning: tricking a shared cache into serving attacker content under a legitimate page's address.

# 12. Coverage Report

**Page-to-section mapping:**

| PDF page(s) | Slide(s) | Guide section |
|---|---|---|
| 1 | 2 | 3.1 |
| 2 | 4, 5 | 3.2, 3.3 |
| 3 | 6, 7 | 3.4, 3.5 |
| 4 | 8, 9 | 3.6, 3.7 |

| PDF page(s) | Slide(s) | Guide section |
|---|---|---|
| 5 | 10, 12 | 3.8, 3.9 |
| 6 | 13, 14 | 3.10, 3.11 |
| 7 | 15, 16 | 3.12, 3.13 |
| 8 | 17, 19 | 3.14, 3.15 |
| 9 | 20, 21 | 3.16, 3.17 |
| 10 | 23, 24 | 3.18, 3.19 |
| 11 | 25, 27 | 3.20, 3.21 |
| 12 | 28, 29 | 3.22, 3.23 |
| 13 | 31, 32 | 3.24, 3.25 |
| 14 | 33, 37 | 3.26, 3.27 |
| 15 | 39, 40 | 3.28, 3.29 |
| 16 | 41, 43 | 3.30, 3.31 |
| 17 | 44, 45 | 3.32, 3.33 |
| 18 | 46, 47 | 3.34, 3.35 |
| 19 | 48, 49 | 3.36, 3.37 |
| 20 | 50 | 3.38 |

**[UNCLEAR] items list:**

1. Page 2, Apache architecture diagram: a box labelled "BMMM Extensible Agent" could not be confirmed; best reading given in section 3.3, labelled uncertain.
2. Page 4, IIS vulnerabilities table: a source URL was cut off, shown only as "https://"; noted in section 3.6 as unresolved.
3. Several screenshots of terminal output (Dirhunt on page 12, Shodan result screen on page 11, Kyubi output on page 14) contain small OCR-garbled fragments inside the raw tool output; the guide reconstructs the readable, meaningful lines and labels the sample outputs in sections 3.21, 3.23, and 3.27 as representative rather than a pixel-exact transcription of every garbled character.

**Items to double check (verify):**

1. The exact CVE number and affected version range for the Apache HTTP/2 "endless CONTINUATION frames" vulnerability (treated in this guide as matching the CVE-2024-27316 class; confirm against the current official EC-Council courseware wording).
2. The exact IIS version at which the metabase configuration file was replaced by `applicationHost.config` (flagged "verify" in section 3.32).

3. The precise, current behavior and feature set of Dirhunt and Fuxploider, since open-source tool behavior can change between versions (flagged "verify" where cited).
4. Whether "FrontJacking" appears by this exact name in the specific official EC-Council CEH v13 courseware text, since this guide confirmed the term from independent security research sources rather than from EC-Council's own publication.

**Coverage confirmation:** Every line item in the inventory file (covering all 20 pages and 38 screenshots) is addressed in Section 3's 38 topic entries, cross-referenced by tool in Section 4, by command in Section 5, by port/protocol in Section 6, by comparison in Section 7, by keyword in Section 8, by question in Section 9, by flashcard in Section 10, and by term in Section 11. No inventory line was left unplaced.